



Air University, Islamabad

**Software Design Description
(SDS DOCUMENT)**

for

EZSell

Version 1.0

By

Ahmed Abbasi 220965

Dawood Haider Chaudhary 221574

Muhammad Ahmed 222550

Supervisor

Dr. Madiha Yousaf

Bachelor of Science in Artificial Intelligence (2022-2026)

Table of Contents

1.	Introduction	1
2.	Design Methodology and Software Process Model	1
2.1	Design Methodology	1
2.2	Software Process Model	1
3.	System Overview	2
3.1	Architectural Design	2
3.2	Module Interaction Flow	3
4.	Design Models.	5
4.1	Activity Diagrams	5
4.2	Class Diagram	9
4.3	System Sequence Diagram	10
4.4	State Transition Diagram	14
5.	Data Design	17
5.1	Data Dictionary	18
6.	Human Interface Design	20
6.1	Screen Images	20
6.2	Screen Objects and Actions	21
7.	Implementation	29
7.1	Algorithm	29
7.2	External APIs/SDKs	36
7.3	User Interface	36
7.4	Deployment	46
8.	Testing and Evaluation	47
8.1	Unit Testing	47
8.2	Functional Testing	48
8.3	Business Rules Testing	52
8.4	Integration Testing	56

Revision History

Name	Date	Reason for changes	Version
Data Collection & NLP Module	08-Dec-2025	Reperformed data scraping process and updated feature extraction using NLP with Regex techniques for improved data quality.	1.1.0
AI Price Prediction Module	12-Dec-2025	Added detailed information about AI models used for price prediction, including model selection, performance metrics, and evaluation results.	1.1.0
Recommendation Module	18-Dec-2025	Implemented and documented the AI-based Recommendation Module for personalized product suggestions.	1.1.0
AR Visualization Module	20-Dec-2025	Added the AR (Augmented Reality) module for 3D furniture visualization and room preview functionality.	1.1.0

Application Evaluation History

Comments (by committee) *include the ones given at scope time both in doc and presentation	Action Taken

Supervised by
Supervisor's Name

Signature_____

1. Introduction

This document describes the design of **EZSell**, an intelligent online marketplace for buying and selling second-hand items, primarily electronics and furniture. The system aims to streamline the resale process by integrating modern technologies such as AI-driven price prediction, analytics dashboards, and user-friendly listing management. It enhances the overall marketplace experience through improved pricing accuracy, transparent user profiles, and insightful activity tracking. So far, the project covers all essential modules required to support a complete second-hand product lifecycle. These core modules include:

- **Profile Management** handles user registration, authentication, verification, and account configuration for both buyers and sellers.
- **Product Listing and Management** enables users to post ads, upload images, edit item details, manage inventory, and control listing visibility.
- **AI-Based Price Prediction** generates fair and accurate price suggestions using machine-learning models based on item condition, features, and current market data.
- **Analytics Dashboard** provides graphical insights and statistics related to user activity, engagement trends, listing performance, and platform usage patterns.

Together, these modules create a smooth workflow from user onboarding to product posting, smart price suggestions, and useful activity insights. This document outlines the key design decisions, system architecture, components, and interfaces that support the development of EZSell.

2. Design Methodology and Software Process Model

2.1 Design Methodology

The system is designed using the Object-Oriented Programming (OOP) approach. Each module of the EZSell platform is modeled as a separate object that encapsulates its own attributes, functions, and responsibilities. For example:

- **The User class** manages authentication, verification, and user roles (Buyer, Seller, Admin).
- **The Listing class** handles product details, images, status updates, and listing lifecycle (create, edit, delete).
- **The PricePredictor class** interacts with the machine-learning model to generate fair price suggestions.

OOP was selected because it promotes modularity, code reusability, and easier long-term maintenance. Each module can be developed, tested, and updated independently without affecting others.

2.2 Software Process Model

The Agile Software Development Model is followed in this project. The system was developed in short, iterative cycles, enabling continuous testing, refinement, and feature enhancement.

Each sprint is focused on a specific module of the EZSell platform. For example, one iteration for **user authentication**, another for **product listing management**, another for **AI-based price prediction**, and subsequent cycles for **AR visualization**, **recommendation features**, and **fraud detection mechanisms**.

Agile was chosen because it supports flexibility, rapid adaptation to user feedback, and incremental integration of advanced components such as AI and AR. This approach ensures smooth progress, reduced development risks, and the ability to enhance modules without restarting the entire development lifecycle.

3. System Overview

EZSell is an AI-powered online marketplace built to simplify the buying and selling of second-hand electronics and furniture. The platform enables sellers to create listings with images and item details, while buyers can browse, search, filter, and contact sellers easily. Intelligent features such as AI-based price prediction, AR furniture visualization, decision-making and improve the overall user experience.

The admin dashboard provides key system insights, including active listings, user engagement, flagged posts, and category-wise trends. Admins can monitor suspicious activity, review reported listings and maintain platform safety using built-in fraud detection tools.

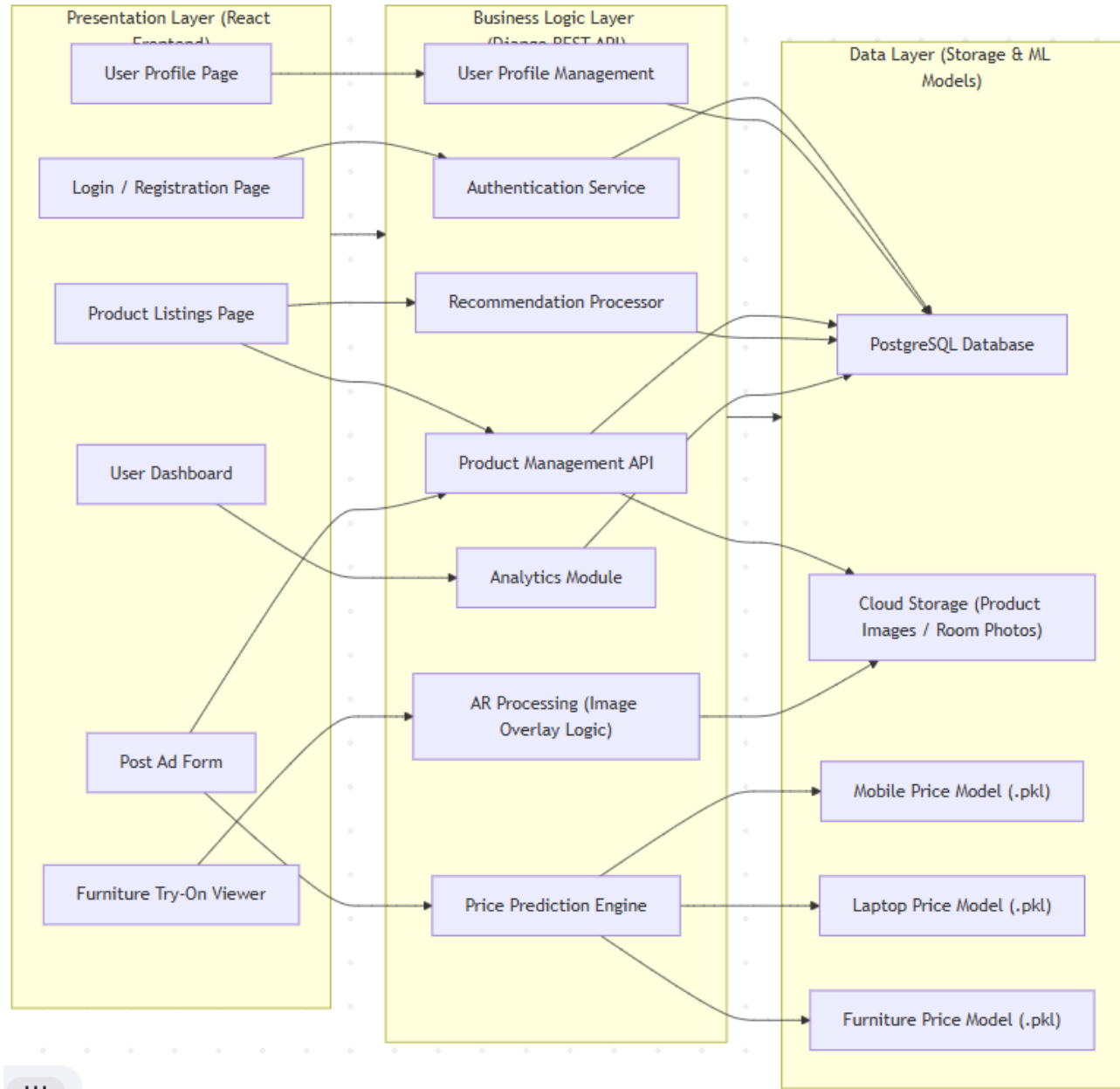
AI-driven recommendations help buyers discover relevant products, and the price prediction engine offers sellers fair pricing suggestions based on item attributes. The AR module allows users to preview used furniture in their own room layout using their device's camera.

Overall, EZSell combines automation, security, and smart technology in a unified marketplace system designed to improve trust, convenience, and usability for all users.

3.1 Architectural Design

The system follows a **Layered Architecture** pattern with three main layers:

1. **Presentation Layer:** User interface components including login and profile pages and product listing screens.
2. **Business Logic Layer:** Core processing modules such as AI-based price prediction, analytics dashboard and AR visualization.
3. **Data Layer:** Databases and storage systems that securely store users, listings, images, pricing models and activity logs.

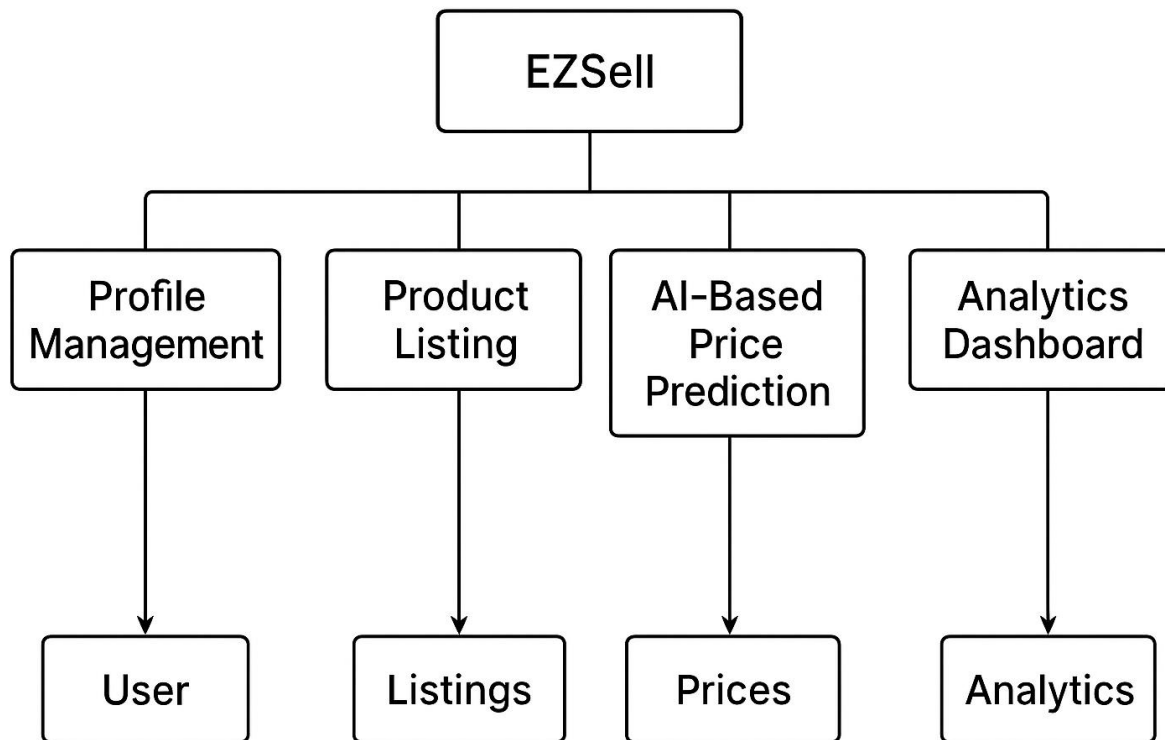


3.2 Module Interaction Flow

1. The user logs in through the Profile Management Module and accesses the platform.
2. Sellers create, edit, or update product listings using the **Product Listings and Management Module**.
3. The **Price Prediction Module** analyzes item details and generates fair price suggestions before a listing is published.
4. Buyers explore listings and use the **AR Customization Module** to preview furniture in their room environment.

5. System activity and listing performance are processed and displayed through the **Analytical Dashboard Module**.

This modular workflow ensures that all components operate together smoothly. The user interface communicates only with the processing layer, while the processing layer interacts with the underlying database. This separation supports better performance, scalability, and simplified system maintenance.



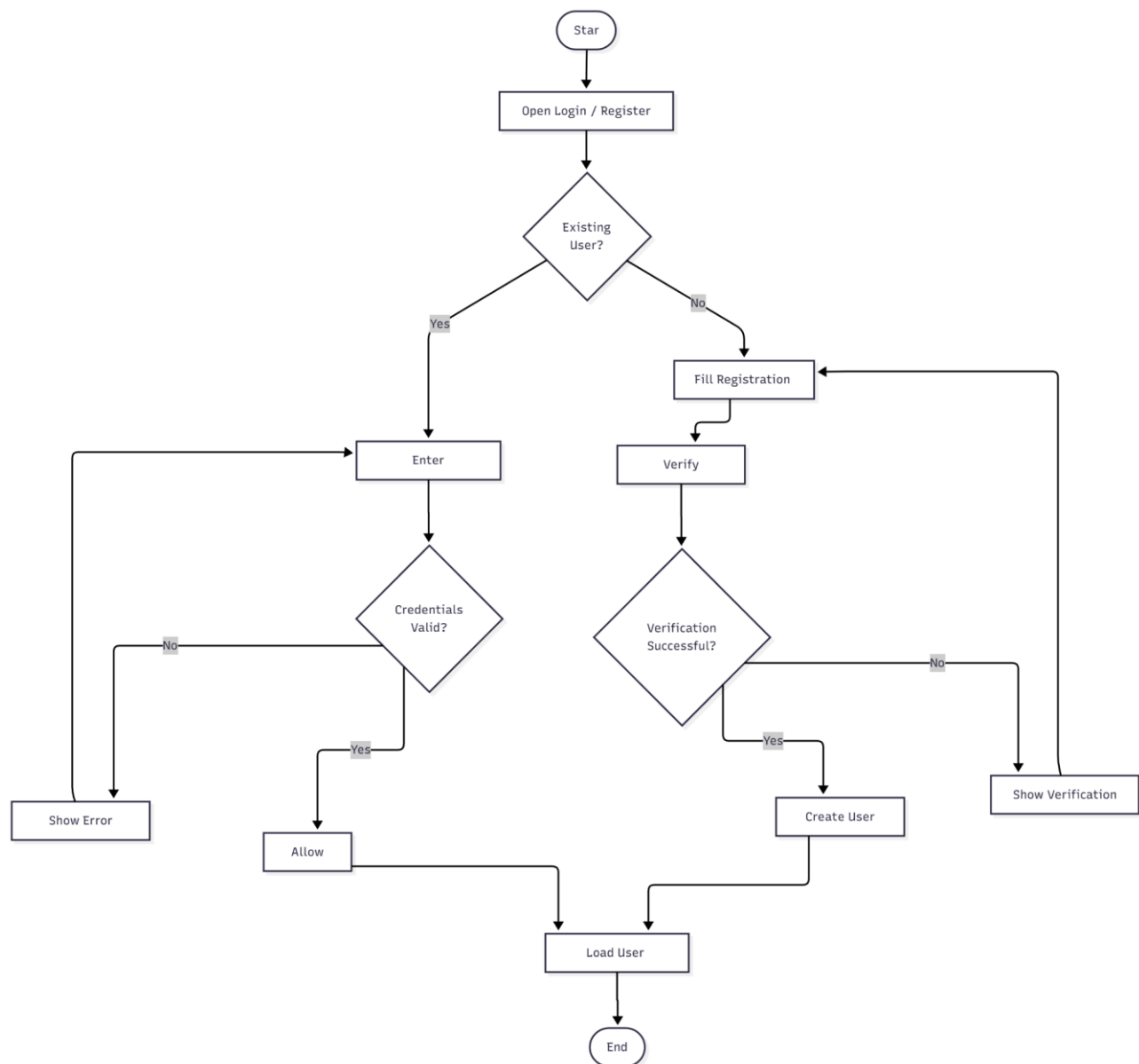
4. Design Models.

Design Models for Object Oriented Development Approach

4.1 Activity Diagrams

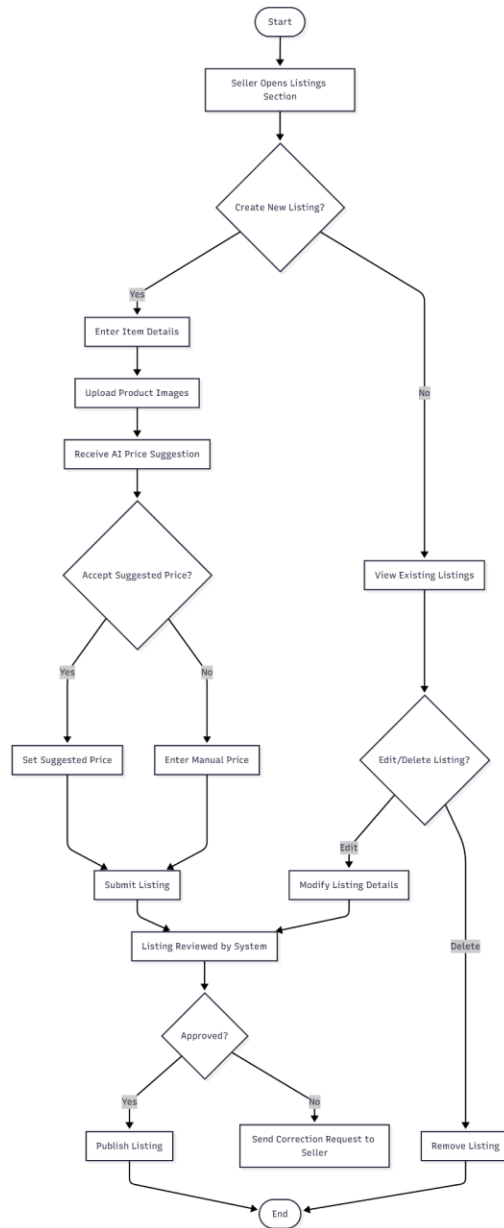
4.1.1 UC-1 Profile Management Module

The user enters login details, the system checks the credentials, and if valid, the user is allowed into the system. If not, an error message is shown.



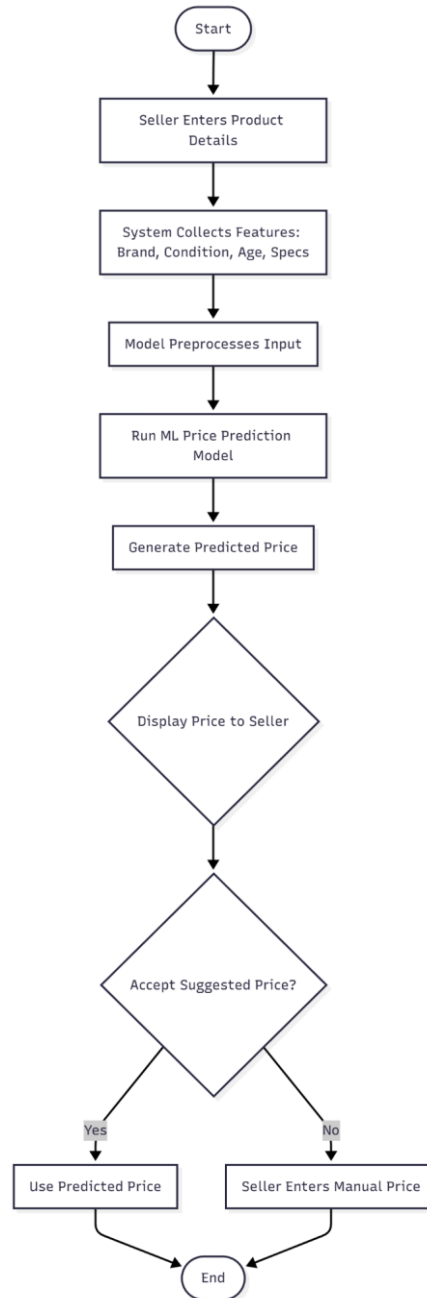
4.1.2 UC-2 Product Listings and Management Module

The seller creates or edits a product listing, the system validates item details, uploads images, applies business rules, and stores the finalized listing in the database.



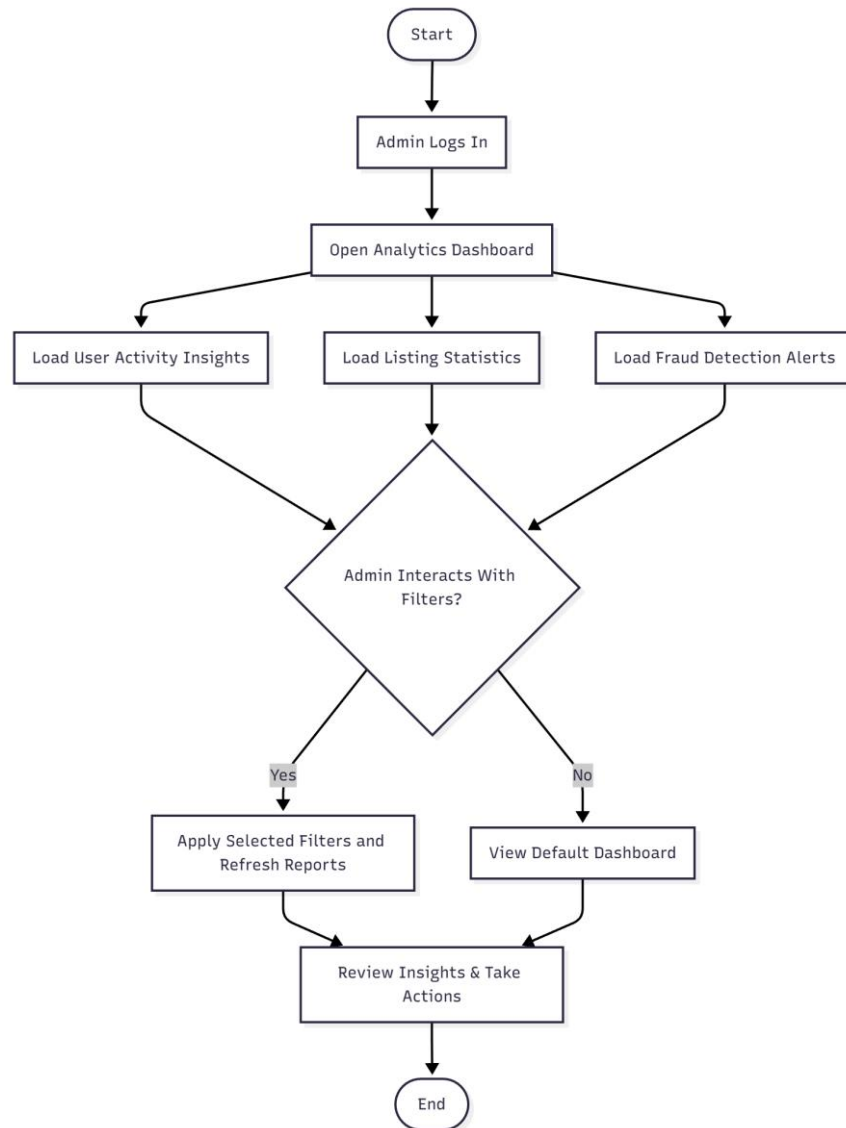
4.1.3 UC-3 Price Prediction Module

The seller enters product specifications, the system processes the attributes, runs the AI pricing model, and returns a fair price suggestion before saving the listing.

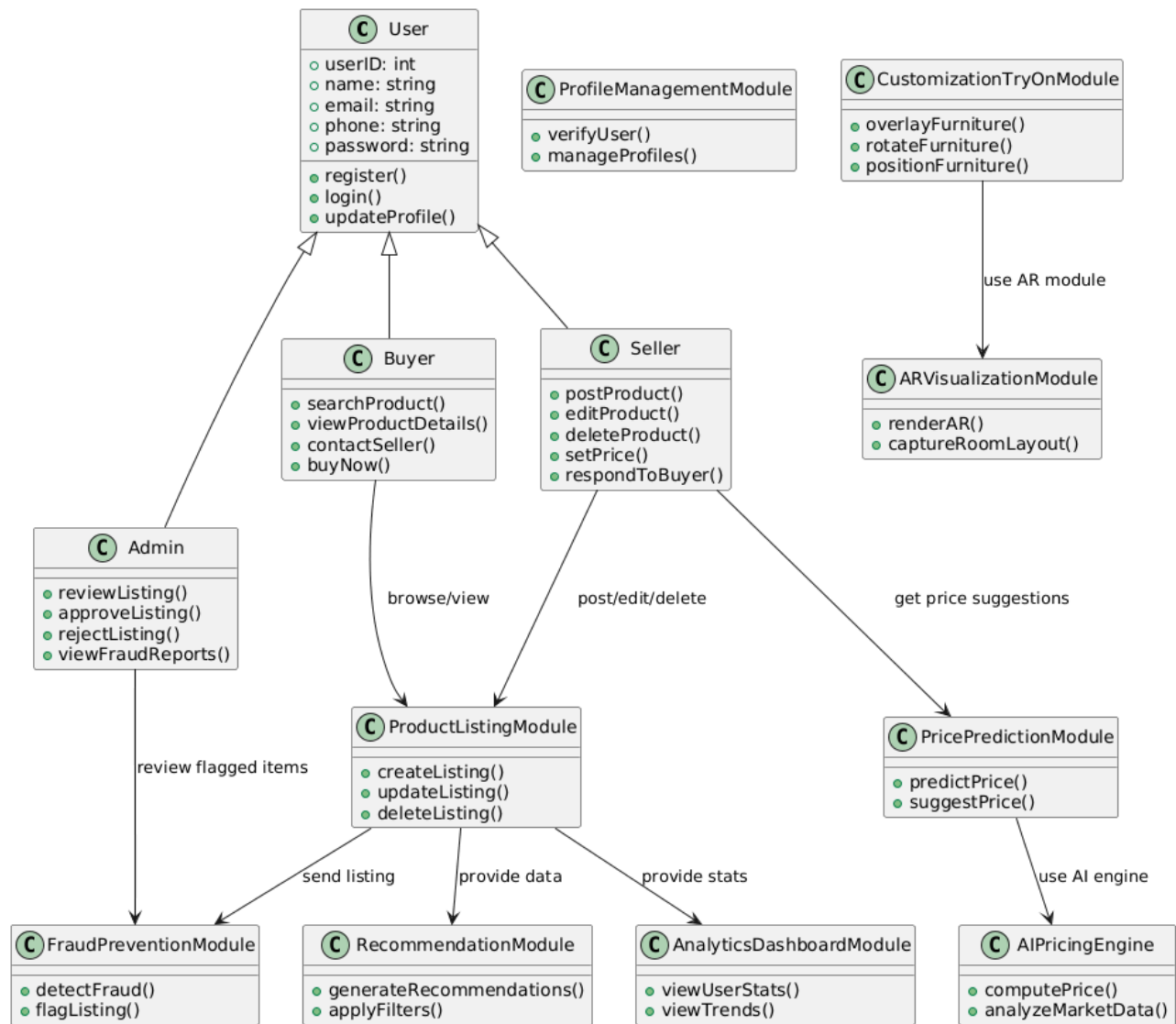


4.1.4 UC-4 Analytical Dashboard

The system collects user activity and listing data, processes trends and metrics, and displays insights such as listing performance and engagement analytics on the dashboard.



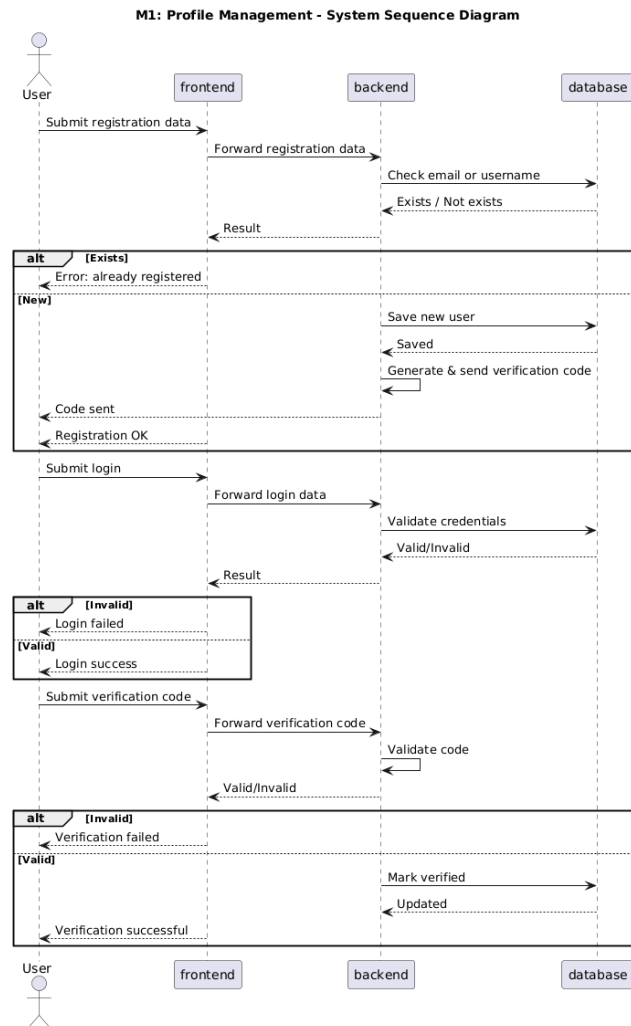
4.2 Class Diagram



4.3 System Sequence Diagram

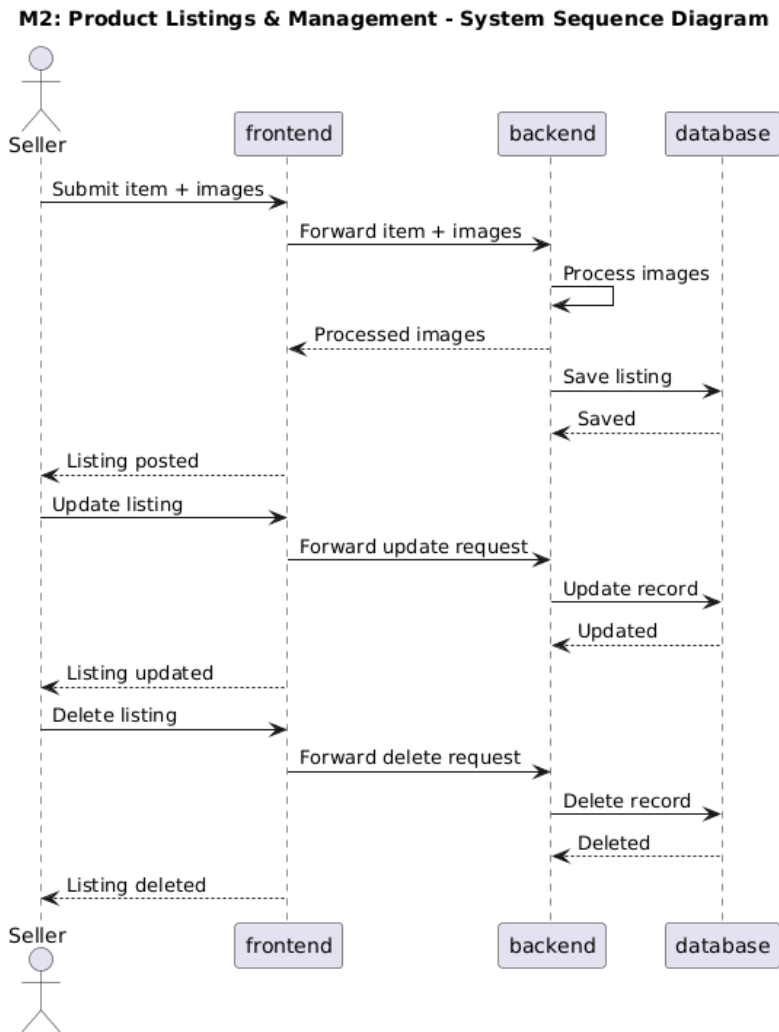
4.3.1 UC-1 Profile Management

Shows how the user sends registration or login inputs to the system, how the system verifies credentials, and how access is granted along with profile data retrieval.



4.3.2 UC-2 Product Listing & Management

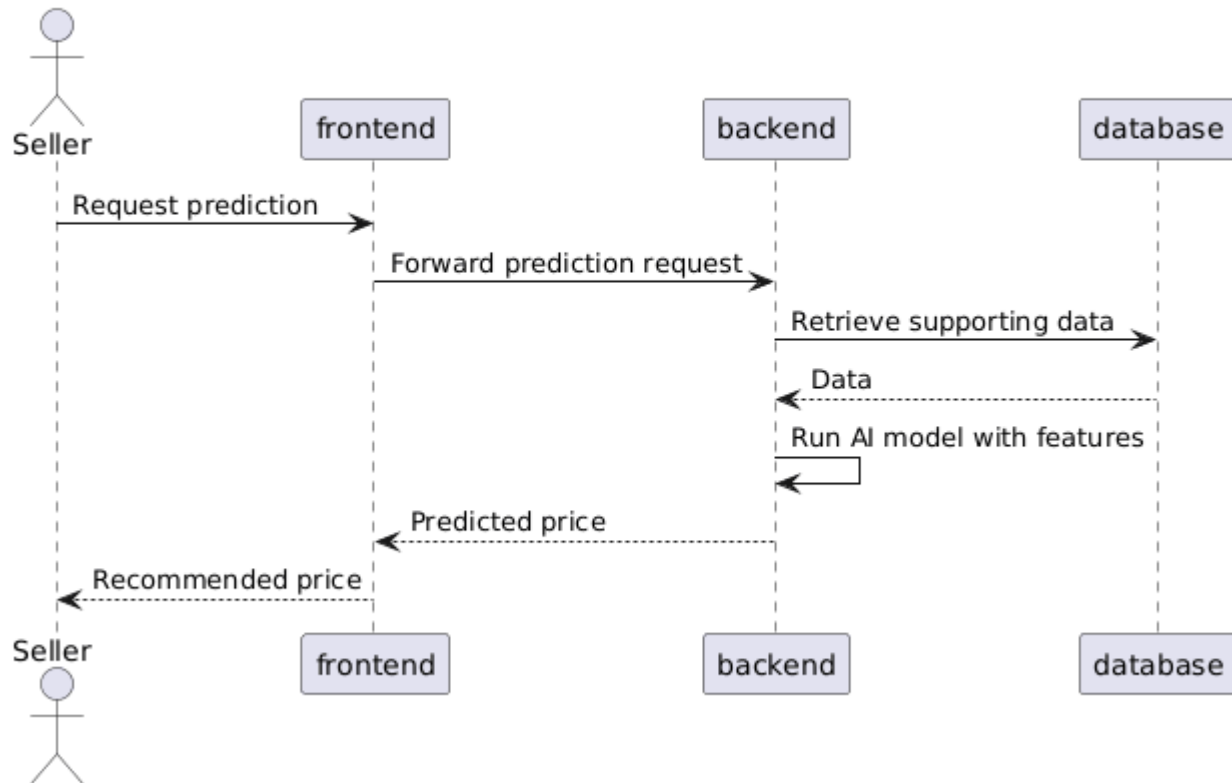
Illustrates how the seller submits product details and images, how the system validates inputs, applies business rules, and stores the listing in the database.



4.3.3 UC -3 Price Prediction

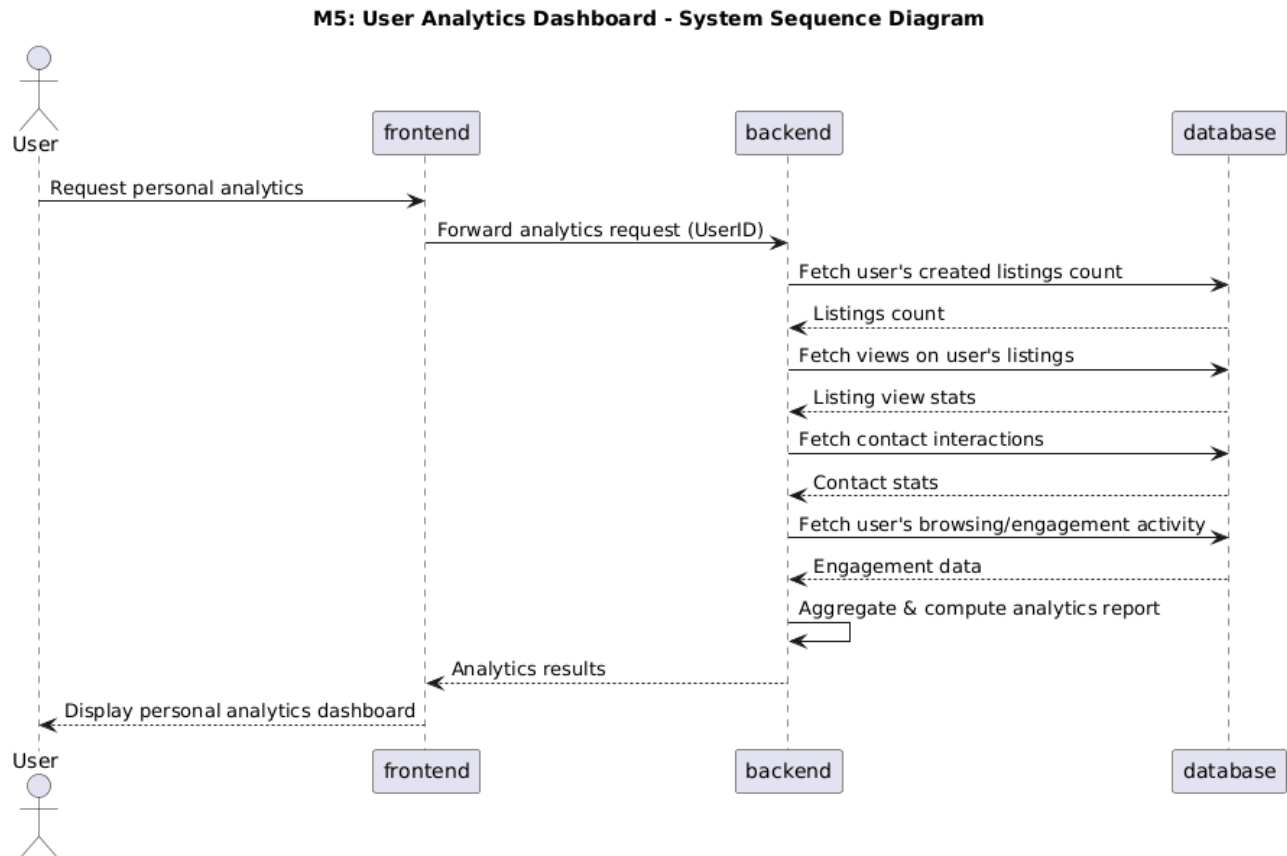
Depicts how the seller provides product specifications, how the system sends the data to the AI pricing engine, receives a suggested price, and updates the listing before publishing.

M3: Price Prediction - System Sequence Diagram



4.3.4 UC-4 Analytical Dashboard

Shows how the system retrieves user and lists data, processes trends and metrics, and displays the insights on the dashboard for buyers, sellers, and admins.

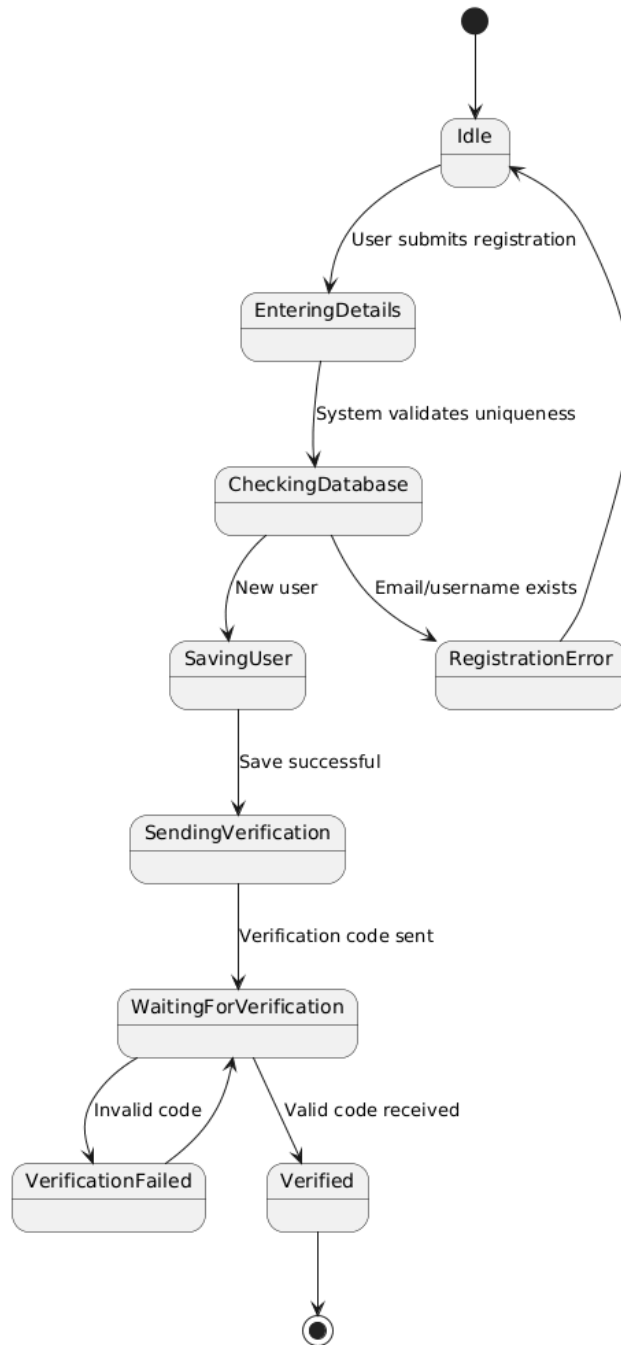


4.4 State Transition Diagram

4.4.1 UC-1 Profile Management

Shows how the system moves from “Logged Out” → “Validating Credentials” → “Logged In” if credentials are correct, or “Error” if validation fails.

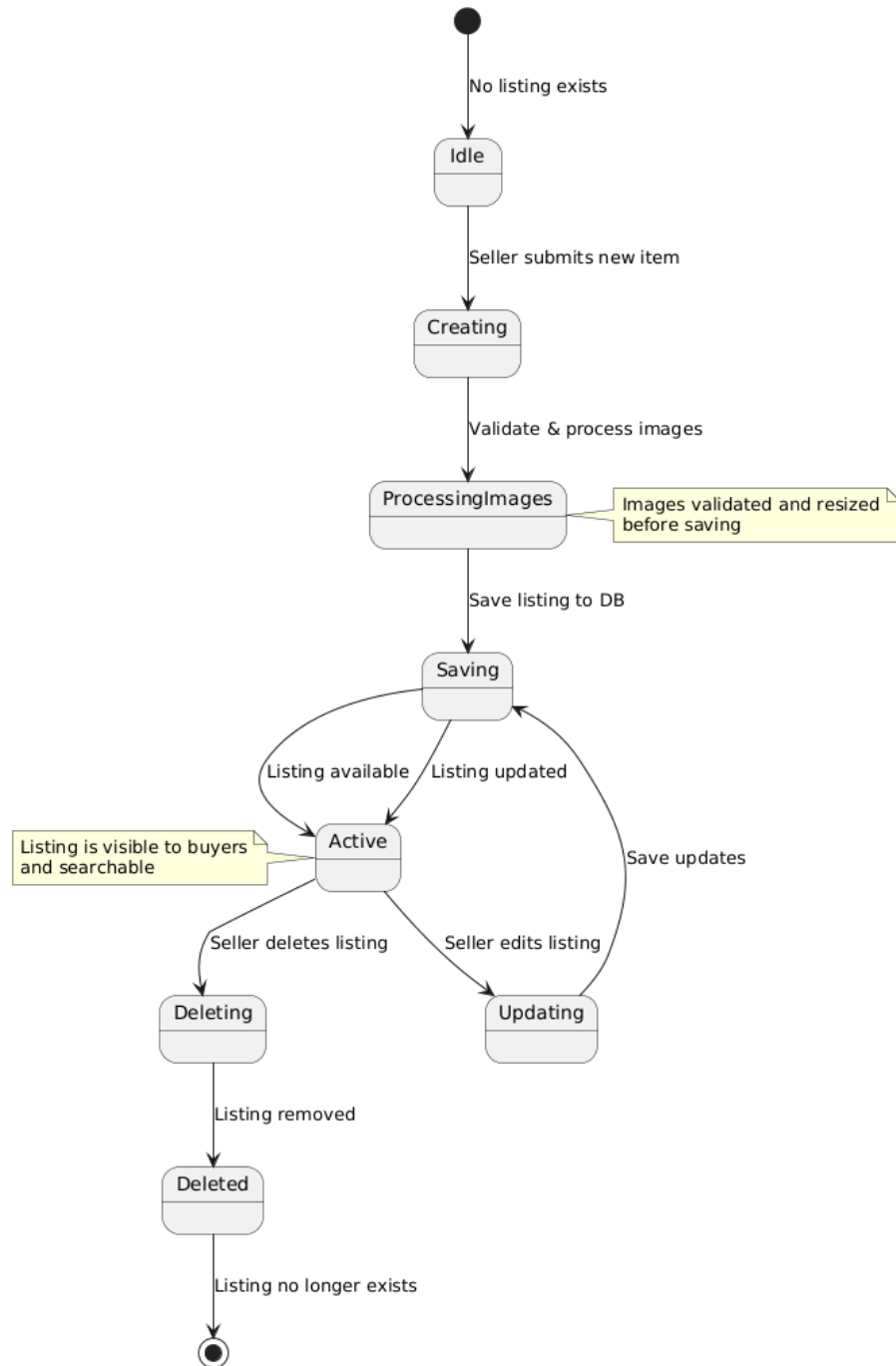
M1: User Registration & Verification - State Diagram



4.4.2 Product Listing & Management

Shows how a listing moves from “Draft” → “Validating Details” → “Published” if all inputs are correct, or “Error/Rejected” if validation fails.

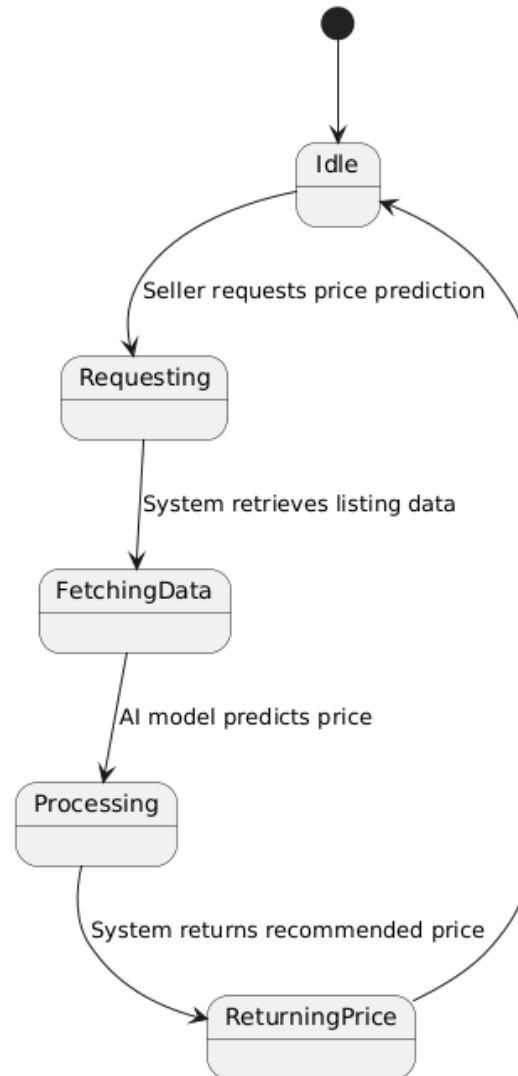
M2: Product Listing Lifecycle - State Diagram



4.4.3 Price Prediction

Shows how a product's pricing moves from "Awaiting Price" → "Predicting Price" → "Price Suggested", or "Manual Input Required" if AI cannot generate a valid suggestion.

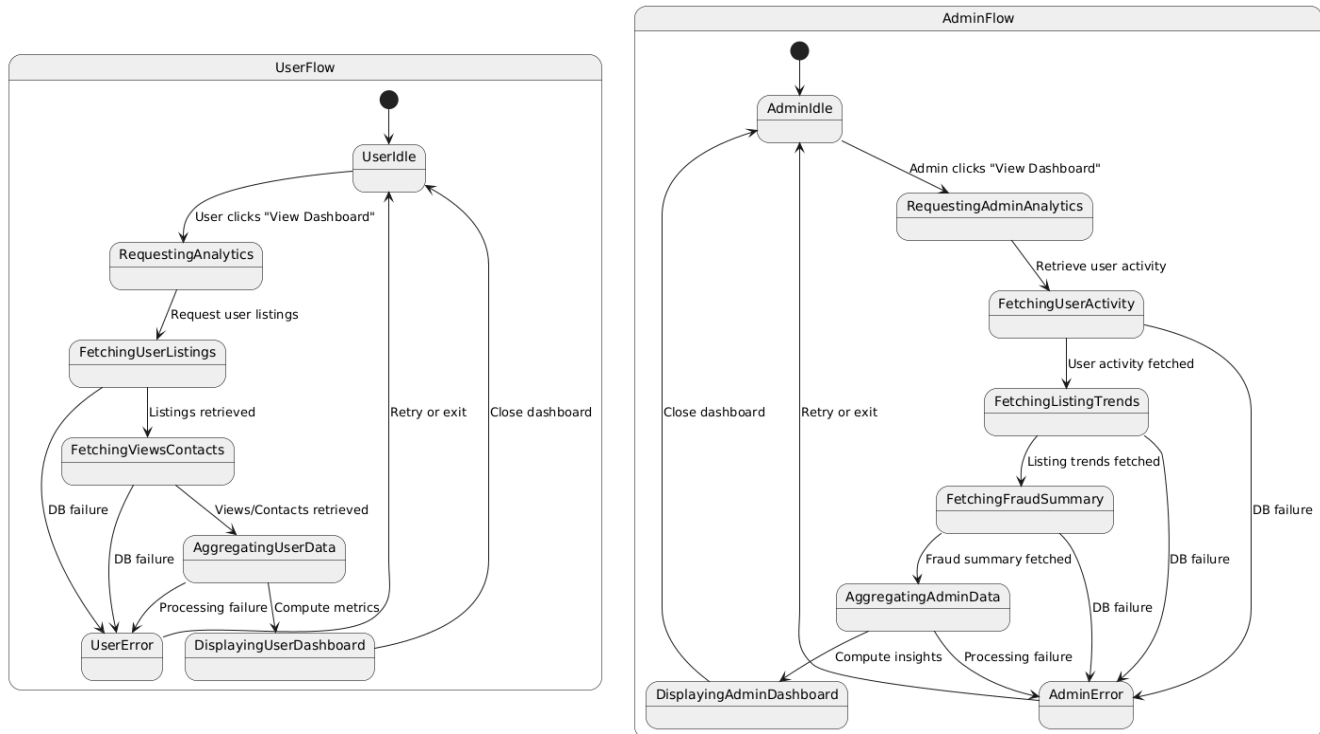
M3: Price Prediction Request - State Diagram



4.4.4 Analytical Dashboard

Shows how dashboard data moves from “Idle” → “Fetching Data” → “Processing Metrics” → “Displaying Insights”.

M5: Analytics Dashboard - Combined State Transition Diagram



5. Data Design

The data design explains how EZSell stores and organizes information. The system handles users, product listings, pricing data, AR previews, and analytics, so the goal is to keep everything structured, secure, and easily accessible.

Database Used:

- PostgreSQL (Supabase Instance)**
 Stores structured data such as user accounts, product listings, AI price suggestions, AR session details, and dashboard analytics. Hosting on Supabase provides managed database services, secure authentication, real-time data syncing, and scalable cloud storage, while

maintaining data integrity, relationships between users and listings, and efficient querying for reporting and recommendations.

Data Flow:

1. Users register or log in; profile information is verified and stored.
2. Sellers create or update product listings, including images and details.
3. The Price Prediction module processes listing attributes and stores suggested prices.
4. Buyers interact with AR module; session details are saved for analytics.
5. Analytics Dashboard reads user activity, listing performance, and AR interactions directly from the database to display insights.

This design ensures efficient data access and keeps all system components synchronized.

5.1 Data Dictionary

5.1.1 User

Represents a platform user who can buy or sell items and access dashboard insights.

Attributes:

- id: int → Unique user identifier
- username: str → Login username
- name: str → Full name
- password_hash: str → Hashed password
- role: str → Buyer, Seller, or Admin

Actions:

- Authenticate user
- Retrieve user information

5.1.2 Product Listing

Represents a second-hand product posted by a seller.

Attributes:

- id: int → Unique listing identifier
- title: str → Item name
- description: str → Item details
- category: str → Product category (electronics, furniture, etc.)
- price: float → Seller-entered or AI-suggested price
- images: list[str] → URLs or paths of uploaded images
- seller_id: int → Owner user ID
- status: str → Draft, Published, or Rejected

Actions:

- Create new listing
- Edit or update listing
- Delete listing

5.1.3 Price Prediction

Represents AI-generated pricing suggestions for listings.

Attributes:

- listing_id: int → Associated product listing
- predicted_price: float → Suggested fair price
- confidence_score: float → Model confidence level
- timestamp: datetime → When prediction was generated

Actions:

- Process listing attributes
- Generate price suggestion
- Store predicted price

5.1.4 Analytical Dashboard

Stores processed metrics and trends for display in dashboards.

Attributes:

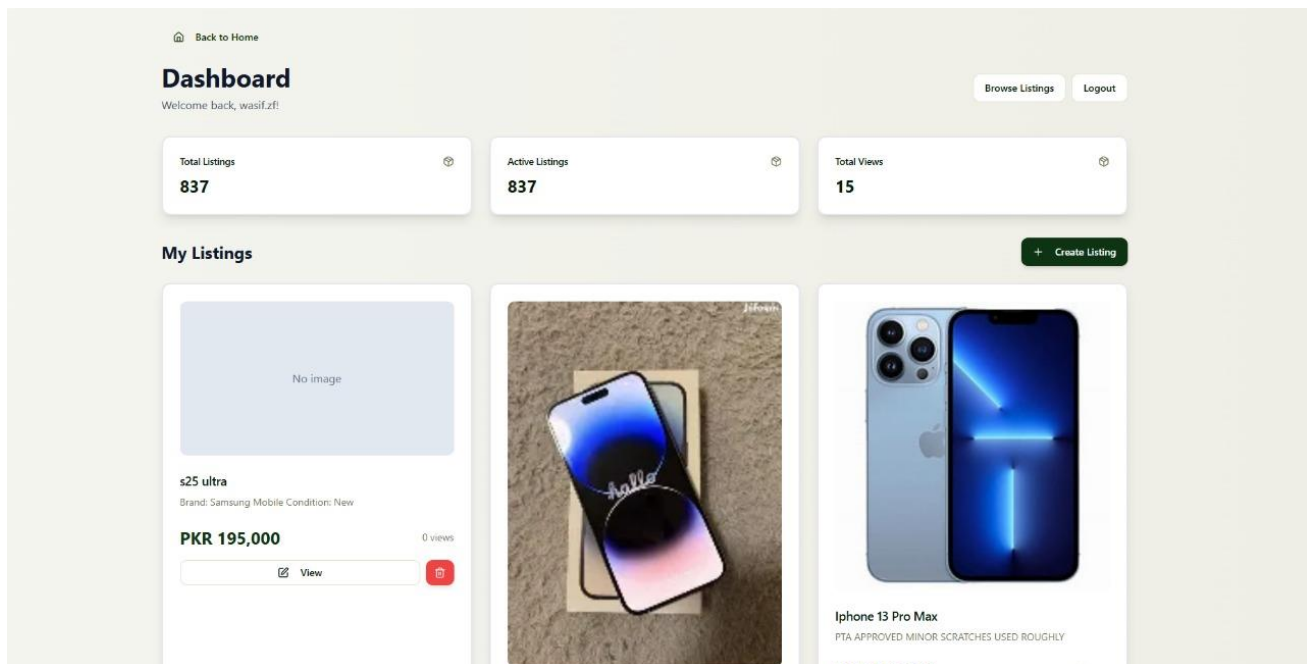
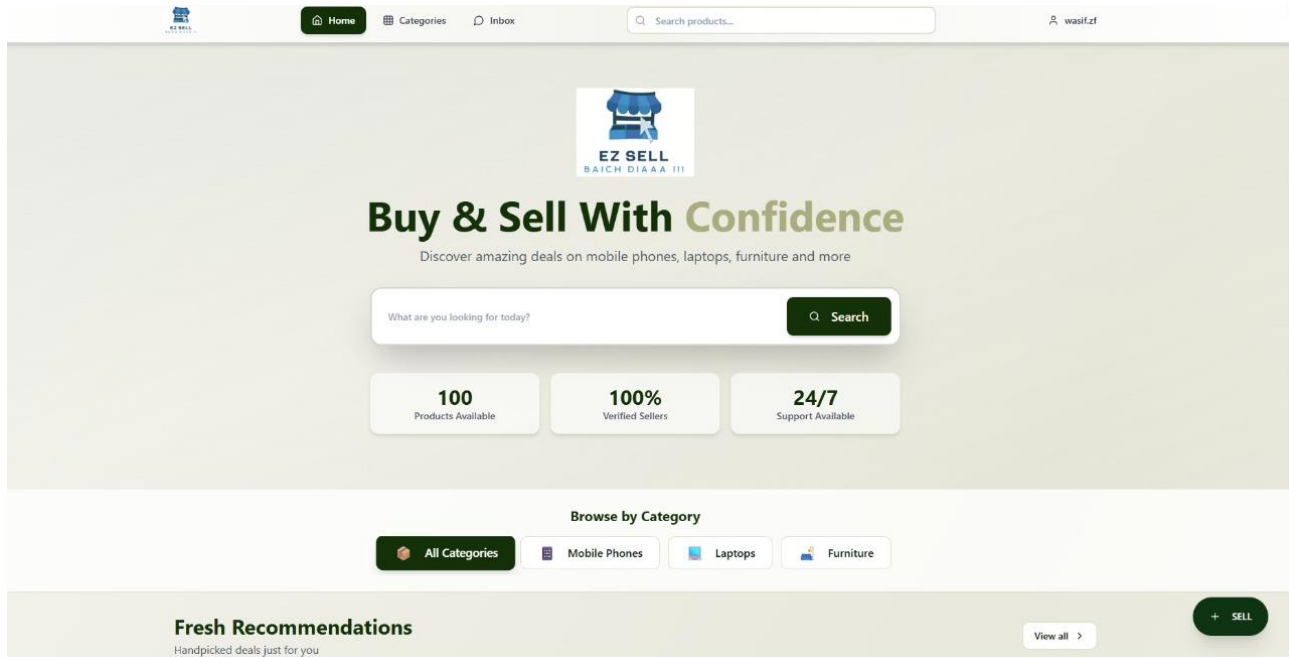
- user_id: int → Associated user
- listing_views: int → Number of views per listing
- sales_count: int → Completed transactions
- engagement_metrics: dict → Likes, favorites, or interactions
- trend_data: dict → Historical performance of listings and categories

Actions:

- Retrieve analytics data
- Calculate trends and metrics
- Display insights on dashboard

6. Human Interface Design

6.1 Screen Images



Create New Listing
Add a new product to your listings

Step 1: Enter Product Details
Fill in the details below to get an AI price prediction

Category *
Mobile

Condition *
Fair

Title * (Include brand, model, specs)
Iphone 13 Pro Max

Description *
PTA APPROVED
MINOR SCRATCHES
USED ROUGHLY

Location
RAJANPUR

Product Image (JPG or PNG)



Step 2: Get AI Price Prediction
Click below to get an intelligent price suggestion based on your product details

Get AI Price Prediction

AI Suggested Price: PKR 135,280.45
Confidence: 73% • Range: 114,988.38 - 155,572.52

Detected Specs:

RAM: 4GB	Storage: 64GB
5G: No	Year: 2023
Tier: Premium	

Step 3: Set Your Price
You can use the AI suggestion or set your own price

Your Price (PKR) *
135280.45

✓ Your price is within acceptable range. Listing will be published immediately.

Create Listing

6.2 Screen Objects and Actions

1. Logo

- **Object Type:** Image / Clickable element
- **Action:** Clicking the EZSell logo returns the user to the homepage from anywhere in the platform.

2. Navigation Bar

- **Home Button:** Navigates to the homepage.
- **Categories Menu:** Opens a dropdown displaying product categories (Mobiles, Laptops, Furniture, etc.).
- **Inbox Button:** Opens the messaging screen for buyer–seller conversations.
- **Search Box:** Allows users to search for products directly from the top navigation bar.
- **User Menu (Top-Right):** Opens profile options including My Listings, Account Settings, and Logout.
-

3. Main Heading + Tagline

- **Object:** Large text section (e.g., “Buy & Sell With Confidence”)
- **Action:** No action; provides branding and user guidance.

4. Central Search Box

- **Text Field:** User enters search queries (e.g., “iPhone 12”).
- **Search Button:** Executes the search and shows relevant listings.

5. Stats Cards

- **Objects Displayed:**
 - Total Products Available
 - Verified Sellers
 - 24/7 Support
- **Action:** Informational only; no clickable actions.

6. Category Filters

- **Objects:** All Categories, Mobile Phones, Laptops, Furniture
- **Action:** Clicking filters the product feed based on category.

7. Floating “Sell” Button

- **Object Type:** Prominent action button (bottom-right corner)
- **Action:** Opens the **Create New Listing** screen.

8. Recommendations Section

- **Product Cards:** Clicking any card opens its detailed product page.
- **View All Button:** Displays the full list of recommended or trending products.

9. Category Dropdown (Create Listing Screen)

- **Object Type:** Dropdown menu
- **Action:** User selects the category for the new listing (e.g., Mobile, Laptop, Furniture).

10. Condition Dropdown

- **Object Type:** Dropdown
- **Action:** User selects the product condition (Fair, Good, Excellent).

11. Title Text Field

- **Object Type:** Text input
- **Action:** User enters a descriptive product title (e.g., “iPhone 13 Pro Max 256GB PTA Approved”).

12. Description Text Area

- **Object Type:** Multi-line text box
- **Action:** User describes item details such as scratches, usage, battery health, etc.

13. Location Text Field

- **Object Type:** Single-line text box
- **Action:** User types the physical location of the item (e.g., Lahore, Rajanpur).

14. Product Image Upload Area

- **Object Type:** Image uploader

- **Actions:**
 - Upload JPG/PNG product images
 - Remove or replace existing images

15. Brand Field

- **Object Type:** Read-only field
- **Action:** Displays detected or selected brand (e.g., Apple, Samsung); no user interaction.

16. “Get AI Price Prediction” Button

- **Object Type:** Action button
- **Action:** Triggers the AI model to estimate the product’s price based on provided details.

17. AI Price Prediction Box

- **Object Type:** Information panel
- **Displays:**
 - Suggested AI price
 - Confidence percentage
 - Detected specifications (RAM, Storage, Year, 5G status, Tier, etc.)
- **Action:** User reviews the suggested price.

18. Price Input Field

- **Object Type:** Numeric text box
- **Action:** User enters the selling price (either accept AI prediction or set their own).

19. Create Listing Button

- **Object Type:** Primary action button
- **Action:** Publishes the product listing after validation of all mandatory fields.

Scrapping Implementation:

This project implemented an automated web scraping system to collect real-world product data from OLX Pakistan marketplace. The system gathered approximately 24,000 product listings across three categories to train machine learning price prediction models.

Scrapping Techniques

1. Selenium WebDriver

Primary method using browser automation to handle JavaScript-rendered content. This technique launches an actual Chrome browser to interact with dynamic web pages.

2. Undetected ChromeDriver

Advanced anti-detection scraping with stealth features including user agent rotation, randomized delays, and bot-detection bypass mechanisms.

3. BeautifulSoup

Lightweight alternative using direct HTTP requests for faster scraping of static content.

Software & Tools Used

- **Selenium** - Browser automation
- **Undetected-chromedriver** - Anti-detection scraping
- **BeautifulSoup4 & lxml** - HTML parsing
- **Pandas & NumPy** - Data processing
- **Requests** - HTTP handling
- **Chrome WebDriver** - Browser control

Data Collection Results

Category	Records Collected
Mobile Phones	5,903
Laptops	11,859
Furniture	6,147
Total	23,909

Scraping Strategy

- **Brand-based targeting:** Scraped specific brands (Apple, Samsung, Dell, HP, etc.)
- **Multi-page crawling:** 20-25 pages per brand
- **Smart extraction:** Automatically extracted RAM, storage, and model details from product titles
- **Quality control:** Price validation (PKR 1,000 - 10,000,000) and duplicate removal

Machine Learning Models Used

1. Random Forest Regressor

- **Type:** Ensemble of decision trees
- **How it works:** Creates multiple decision trees using random subsets of data, each tree makes predictions, and final prediction is the average of all trees
- **Performance:** 89.14% R^2 for laptops

2. Gradient Boosting Regressor

- **Type:** Sequential error correction
- **How it works:** Builds trees sequentially where each new tree corrects errors made by previous trees
- **Performance:** 68.99% R^2 for laptops, **99.94% R^2 for mobiles, 99.96% R^2 for furniture**

3. Extra Trees (Extremely Randomized Trees)

- **Type:** Randomized forest
- **How it works:** Similar to Random Forest but with more randomization in node splitting
- **Performance:** 92.36% R^2 for laptops

4. XGBoost (Extreme Gradient Boosting)

- **Type:** Optimized gradient boosting
- **How it works:** Enhanced gradient boosting with regularization (L1/L2), handles missing values natively, uses parallel tree construction
- **Performance:** 92.45% R^2 baseline → 92.29% R^2 after tuning for laptops

5. LightGBM

- **Type:** Fast gradient boosting
- **How it works:** Uses histogram-based algorithms for faster training and grows trees leaf-wise
- **Performance:** 87.28% R^2 for laptops

Evaluation Metrics

Category	Model	Train R^2	Test R^2	MAE (Rs.)	RMSE (Rs.)	MAPE
Laptop	XGBoost (Tuned)	95.12%	92.29%	1,702	6,911	3.60%
Laptop	Extra Trees	94.87%	92.36%	1,745	6,958	3.72%
Laptop	XGBoost (Baseline)	93.21%	92.45%	1,689	6,895	3.58%
Laptop	Random Forest	91.45%	89.14%	2,134	8,245	4.51%
Laptop	LightGBM	89.73%	87.28%	2,567	8,932	5.23%
Laptop	Gradient Boosting	72.18%	68.99%	4,328	13,967	8.91%
Laptop	Stacked Ensemble	93.89%	91.76%	1,823	7,201	3.85%
Mobile	Gradient Boosting	99.96%	99.94%	168	602	0.82%
Mobile	XGBoost	99.95%	99.93%	175	623	0.87%
Mobile	LightGBM	99.93%	99.91%	189	678	0.94%
Mobile	Random Forest	99.89%	99.85%	231	845	1.12%

Category	Model	Train R ²	Test R ²	MAE (Rs.)	RMSE (Rs.)	MAPE
Mobile	Extra Trees	99.87%	99.83%	245	892	1.21%
Furniture	Gradient Boosting	99.98%	99.96%	109	357	0.31%
Furniture	XGBoost	99.97%	99.95%	118	385	0.34%
Furniture	LightGBM	99.96%	99.94%	127	412	0.38%
Furniture	Extra Trees	99.94%	99.92%	145	468	0.45%
Furniture	Random Forest	99.92%	99.89%	163	521	0.52%

Best Performing Models & Reasons

For Laptops: XGBoost (92.29% R²)

Why it worked best:

1. **Regularization (L1/L2):** Built-in penalties prevent overfitting on small dataset (1,269 samples with 44 features)
2. **Missing Data Handling:** Native sparse-aware algorithm crucial since laptop data had incomplete RAM/storage information
3. **Feature Interaction Capture:** Uses second-order gradients to better capture complex relationships like processor-GPU combinations
4. **Hyperparameter Optimization: Grid search found optimal parameters:**
 - max_depth: 10
 - learning_rate: 0.1
 - n_estimators: 300
 - subsample: 0.7
 - colsample_bytree: 0.7
5. **Better than alternatives:**
 - Beat Extra Trees (92.36%)
 - Significantly better than vanilla Gradient Boosting (68.99%)
 - More accurate than Stacked Ensemble (91.76%)

For Mobile & Furniture: Gradient Boosting (99.94% & 99.96% R²)

Why it worked best:

1. **Larger Dataset:** Mobile had 5,963 samples, Furniture had 1,691 samples (more data = less overfitting risk)
2. **Simpler Feature Space:** Only 10 features compared to laptops' 44 features
3. **Sequential Error Correction:** Additive model systematically reduced residual errors to near-perfect accuracy
4. **Lower Missing Data:** <5% missing values meant XGBoost's advanced missing data handling wasn't needed
5. **Computational Efficiency:** Faster to train and deploy than XGBoost for these simpler problems

Recommendations Module:

The EzSell platform features an AI-powered Recommendation System that delivers personalized product suggestions through behavioral analysis and machine learning algorithms.

Core Components

1. Activity Tracking

Captures user interactions (searches, views, favorites, messages) for both authenticated and anonymous users, with automatic keyword extraction using NLP.

2. Interest Profiling

Builds dynamic user profiles based on preferred categories, brands, keywords, and price ranges, updating in real-time.

3. Recommendation Engine

- Generates personalized suggestions using:
- Interest-based recommendations (user history)
- Similar item matching (content-based filtering)
- Trending products (popularity-based)
- Collaborative filtering (keyword similarity)

Technical Details

- **Scoring:** Weighted algorithm combining category (40%), keywords (40%), price (10%), and brand (10%) matching
- **APIs:** 15+ RESTful endpoints for tracking, recommendations, and analytics
- **Analytics:** Interactive dashboard with engagement metrics, CTR tracking, and activity timelines
- **Privacy:** User-controlled history with clear/export options

The system enhances product discovery through personalized feeds while providing actionable insights for sellers.

AR Implementation

Overview

The Advanced 3D AR Preview is a web-based augmented reality feature that allows users to visualize furniture items in their own rooms before purchase. The system enables customers to upload photos of their spaces and virtually place multiple furniture pieces with customizable attributes.

Core Functionality

The system operates through a three-stage user workflow:

1. Room Upload: Users capture or upload photographs of their intended space
2. Interactive Customization: Multiple furniture items can be placed with adjustable size (50%-200%), rotation (0-360°), and color options
3. Preview & Save: Final arrangements are reviewed and stored for future reference

Technical Implementation

Frontend Technology:

- Built using React.js with TypeScript for type safety
- HTML5 Canvas API for real-time 2D rendering with 3D visual effects
- Radix UI components for professional interface elements
- Responsive design compatible with desktop and mobile devices

Backend Architecture:

- FastAPI framework for RESTful API endpoints
- OpenCV library for image processing and overlay generation
- PostgreSQL database for metadata storage
- File-based storage system for generated AR previews

Key Features

- Multi-item placement with individual customization controls
- Real-time visual feedback during manipulation
- 3D-like effects including shadows, gradients, and depth perception
- Color palette selection (6 preset options)
- Persistent storage of user-generated arrangements
- Category-specific activation (furniture listings only)

Dependencies

Frontend: React 18.3, Axios, Radix UI, Tailwind CSS, Vite

Backend: FastAPI, OpenCV, NumPy, Pillow, Uvicorn

7. Implementation

7.1 Algorithm

Algorithm 1: User Login
Input: email, password
Output: "Login Successful" OR "Invalid email" OR "Incorrect Password", session.user_id (if login successful)
<pre> 1: email_input ← INPUT("Enter Email") 2: password_input ← INPUT("Enter Password") 3: 4: userRecord ← FETCH User WHERE email = email_input 5: 6: if (userRecord = NULL) then 7: DISPLAY "Invalid email" 8: EXIT 9: end if 10: 11: if (HASH(password_input) = userRecord.password_hash) then 12: session.user_id ← userRecord.user_id 13: DISPLAY "Login Successful" 14: else 15: DISPLAY "Incorrect Password" 16: end if </pre>
Algorithm 2 — Seller Dashboard
Input: seller choices: "View Listings", "Add New", "Edit", "Delete"
Output: seller dashboard actions (listings shown, edited, deleted, or new ad posted)
<pre> 1: DISPLAY "Seller Dashboard" 2: 3: CHOICE ← INPUT("Choose: View Listings / Add New / Edit / Delete") 4: 5: if (CHOICE = "View Listings") then 6: SHOW all listings WHERE seller_id = session.user_id 7: end if 8: 9: if (CHOICE = "Add New") then 10: CALL PostAdProcedure() 11: end if 12: 13: if (CHOICE = "Edit") then 14: UPDATE selected listing fields 15: end if 16: 17: if (CHOICE = "Delete") then 18: DELETE selected listing 19: end if </pre>

Algorithm 3.1: EZSell detectDuplicateListings	Algorithm 3.2: EZSell detectPriceMismatch
Input: user_id: integer	Input: listing_id: integer, user_price: float
Output: duplicates: Set of flagged duplicate listings	Output: price_flag: Dictionary or NULL
Algorithm 3.1: EZSell detectDuplicateListings Input: user_id: integer Output: duplicates: List[Dictionary] <pre> 1: duplicates ← [] 2: listings ← queryListings(user_id) 3: seen_titles ← {} 4: for each ad in listings do 5: normalized_title ← toLowerCase(trim(ad.title)) 6: if normalized_title in seen_titles then 7: duplicates.append({ 8: type: "duplicate_title", 9: listing_id: ad.id, 10: duplicate_of: seen_titles[normalized_title] 11: }) 12: else 13: seen_titles[normalized_title] ← ad.id 14: end if 15: end for 16: return duplicates </pre>	Algorithm 3.2: EZSell detectPriceMismatch Input: listing_id: integer, user_price: float Output: price_flag: Dictionary or NULL <pre> 1: model ← loadPricePredictionModel() 2: listing_features ← fetchListingFeatures(listing_id) 3: predicted_price ← model.predict(listing_features) 4: lower_bound ← predicted_price * 0.70 // 30% below market 5: upper_bound ← predicted_price * 1.30 // 30% above market 6: if user_price < lower_bound OR user_price > upper_bound then 7: return { 8: type: "price_out_of_range", 9: listing_id: listing_id, 10: predicted_price: predicted_price, 11: user_price: user_price, 12: action: "send_to_admin_review" 13: } 14: end if 15: return NULL </pre>

Algorithm 4: Post Ad Procedure (Formatted)

Input: title, description, category, price, images (multiple images)

Output: listing created, listing images saved. confirmation message

```
1:  DISPLAY "Seller Dashboard"
2:
3:  CHOICE ← INPUT("Choose: View Listings / Add New / Edit / Delete")
4:
5:  if (CHOICE = "View Listings") then
6:      SHOW all listings WHERE seller_id = session.user_id
7:  end if
8:
9:  if (CHOICE = "Add New") then
10:     CALL PostAdProcedure()
11:  end if
12:
13: if (CHOICE = "Edit") then
14:     UPDATE selected listing fields
15: end if
16:
17: if (CHOICE = "Delete") then
18:     DELETE selected listing
19: end if
```

Algorithm 5: Price Prediction Process (Formatted)

Input: listing_features (brand, specs, year, condition, etc.), trained model

Output: predicted_price. confidence score, prediction saved to database

```
1:  listing_features ← INPUT("Enter listing features")
2:
3:  model ← LOAD trained_price_prediction_model
4:
5:  predicted_price ← model.predict(listing_features)
6:
7:  confidence ← model.confidence_score(predicted_price)
8:
9:  SAVE prediction(listing_id, predicted_price, confidence)
10:
11: DISPLAY predicted_price, confidence
```

Algorithm 6: User Dashboard (Formatted)

Input: session.user_id

Output: report: analytics chart (listings count, views, contacts, last active)

```

1: user_id ← session.user_id
2:
3: total_listings ← COUNT listings WHERE user_id = user_id
4: total_views ← SUM views WHERE user_id = user_id
5: total_contacts ← COUNT messages WHERE seller_id = user_id
6: last_active ← FETCH latest login timestamp
7:
8: DISPLAY analytics_chart(total_listings, total_views, total_contacts, last_active)

```

Algorithm 7: Admin Dashboard (Formatted)

Input: invoices: none (system retrieves all data)

Output: admin charts (users, listings, fraud, active listings), full summary dashboard

```

1: total_users ← COUNT all users
2: total_listings ← COUNT all listings
3: active_listings ← COUNT listings WHERE status = "active"
4: fraud_alerts ← COUNT fraud_alerts
5:
6: chart1 ← CREATE user_growth_chart()
7: chart2 ← CREATE listing_trend_chart()
8: chart3 ← CREATE fraud_statistics_chart()
9:
10: DISPLAY dashboard WITH chart1, chart2, chart3, and summary metrics

```

Algorithm 8: Recommendation

INPUT: user_id, limit, exclude_viewed

OUTPUT: List of (listing, relevance_score, recommendation_type)

```

1. Query all UserActivity records for user_id
2. Extract from activities:
   - Categories (mobile, laptop, furniture)
   - Keywords (brand names, specs, features)
   - Brands (Samsung, iPhone, Dell, etc.)
   - Price ranges (min, max from viewed items)

3. Aggregate using frequency counting:
   category_counts = Counter([activity.category for activity in activities])
   keyword_counts = Counter([keyword for keywords in activities])
   brand_counts = Counter([brand for brands in activities])

4. Store in UserInterest table:
   - categories: JSON {"mobile": 15, "laptop": 8}
   - keywords: JSON {"samsung": 10, "gaming": 5, "laptop": 8}
   - brands: JSON {"Samsung": 7, "Dell": 4}
   - price_range_min, price_range_max

```

```

Query active, approved listings
2. Exclude:
  - User's own listings
  - Already viewed listings (if exclude_viewed = True)

3. Filter:
WHERE is_active = TRUE
  AND is_approved = TRUE
  AND listing_id NOT IN (viewed_ids, own_listing_ids)

FOR each listing in candidates:
  score = 0.0

  // Category Match (40% weight)
  IF listing.category IN user_categories:
    category_frequency = user_categories[listing.category]
    total_category_views = sum(user_categories.values())
    score += (category_frequency / total_category_views) × 0.4

  // Keyword Match (40% weight)
  listing_keywords = extract_keywords(listing.title + listing.description)
  keyword_matches = 0
  FOR each keyword IN listing_keywords:
    IF keyword IN user_keywords:
      keyword_matches += user_keywords[keyword]
  total_keyword_frequency = sum(user_keywords.values())
  score += (keyword_matches / total_keyword_frequency) × 0.4

  // Price Range Match (10% weight)
  IF user_price_min ≤ listing.price ≤ user_price_max:
    score += 0.10
  ELSE IF listing.price is close to range (within 30%):
    score += 0.05

  // Brand Match (10% weight)
  IF listing.brand IN user_brands:
    score += 0.10

  // Recency Bonus
  days_old = (current_date - listing.created_at).days
  IF days_old < 7:
    score × 1.1 // 10% boost for new listings

  IF score > 0.1: // Minimum threshold
    recommendations.append((listing, score, "interest_based"))
1. Sort recommendations by score (descending)
2. Return top 'limit' results
3. Save to RecommendationHistory for tracking

FALLBACK: If no user interests exist → Return trending listings

```

Algorithm 9: 3D AR Preview

INPUT: room_image (photo), furniture_title, price, placement_data

OUTPUT: AR preview image with furniture overlay + room analysis

```

1 Read uploaded room image → Convert to numpy array
2. Convert color spaces:
  - gray = cv2.cvtColor(img, COLOR_BGR2GRAY)
  - hsv = cv2.cvtColor(img, COLOR_BGR2HSV)

3. Edge Detection (Find walls/floor):
  edges = cv2.Canny(gray, threshold1=50, threshold2=150)
  lines = cv2.HoughLinesP(edges, rho=1, theta=π/180, threshold=100)

4. Estimate Room Dimensions:
  aspect_ratio = image_width / image_height
  estimated_width = 4.0 + (aspect_ratio - 1) × 2 // 4-6 meters
  estimated_depth = 4.5 + (1 / aspect_ratio) × 1.5 // 4.5-6 meters
  estimated_height = 2.7 + random(0, 0.4) // 2.7-3.1 meters

5. Dominant Color Analysis (K-Means Clustering):
  pixels = img.reshape(-1, 3) // Flatten to 1D array of RGB values
  k_means = cv2.kmeans(pixels, k=5, iterations=100)
  dominant_colors = k_means.cluster_centers
  color_frequencies = count(k_means.labels)

6. Lighting Detection:
  avg_brightness = mean(gray_pixels)
  IF avg_brightness > 120:
    lighting = "Natural (Window)"
  ELSE:
    lighting = "Artificial (Ceiling)"

7. Floor Type Detection (Bottom 1/3 of image):
  floor_region = gray[height × 0.66 : height, :]
  floor_texture_std = std_deviation(floor_region)

  IF floor_texture_std < 20:
    floor_type = "Tile"
  ELIF floor_texture_std < 35:
    floor_type = "Laminate"
  ELIF floor_texture_std < 50:
    floor_type = "Hardwood"
  ELSE:
    floor_type = "Carpet"

8. Wall Color Detection (Top 1/3 of image):
  wall_region = img[0 : height × 0.33, :]
  wall_avg_color = mean(wall_region, axis=(0,1))
  wall_color = match_closest_color(wall_avg_color, WALL_COLOR_PALETTE)

9. Style Classification:
  dominant_colors_match = compare_colors(dominant_colors, ROOM_STYLE_PALETTES)
  room_style = argmax(dominant_colors_match) // "Modern", "Scandinavian", etc.

FOR each furniture_item in placement_data:
  // Get placement parameters
  x = item.x_position
  y = item.y_position
  scale = item.scale // 1.0 = default size
  rotation = item.rotation // degrees
  color = item.color // RGB tuple

  // Calculate furniture dimensions
  furniture_width = 200 × scale
  furniture_height = 150 × scale
  depth_offset = 80 × scale

  // SHADOW RENDERING (Ellipse below furniture)
  shadow_center = (x, y + furniture_height/2 + 10)
  shadow_size = (furniture_width/2, furniture_height/6)
  cv2.ellipse(img, shadow_center, shadow_size,
    angle=0, start_angle=0, end_angle=360,

```

```

        color=(0,0,0), thickness=-1)
    apply_alpha_blend(img, shadow_opacity=0.3)

    // MAIN FRONT FACE (Rectangle)
    top_left = (x - furniture_width/2, y - furniture_height/2)
    bottom_right = (x + furniture_width/2, y + furniture_height/2)
    cv2.rectangle(img, top_left, bottom_right, color, thickness=-1)

    // SIDE PANEL (Polygon for depth illusion)
    side_points = [
        (bottom_right.x, top_left.y),
        (bottom_right.x + depth_offset, top_left.y - depth_offset),
        (bottom_right.x + depth_offset, bottom_right.y - depth_offset),
        (bottom_right.x, bottom_right.y)
    ]
    side_color = color * 0.7 // 30% darker for depth
    cv2.fillPoly(img, [side_points], side_color)

    // TOP PANEL (Polygon for height)
    top_points = [
        (top_left.x, top_left.y),
        (top_left.x + depth_offset, top_left.y - depth_offset),
        (bottom_right.x + depth_offset, top_left.y - depth_offset),
        (bottom_right.x, top_left.y)
    ]
    top_color = color * 1.2 // 20% lighter for highlight
    cv2.fillPoly(img, [top_points], top_color)

    // BORDER (Green outline for selection)
    cv2.rectangle(img, top_left, bottom_right, (0,255,0), thickness=3)

    // LABEL (Furniture title below item)
    label_background = (top_left.x, bottom_right.y + 5, furniture_width, 25)
    cv2.rectangle(img, label_background, (20,49,9), thickness=-1)
    cv2.putText(img, furniture_title,
        position=(top_left.x + 5, bottom_right.y + 20),
        font=HERSHEY_SIMPLEX, size=0.5, color=(255,255,255))

recommendations = []

// Fit Recommendation (based on dimensions)
room_area = estimated_width * estimated_depth
IF furniture fits comfortably in room_area:
    confidence = 90 + min(5, suitability_bonus)
    recommendations.append({
        "type": "fit",
        "message": f"Furniture fits well ({estimated_width}m x {estimated_depth}m room)",
        "confidence": confidence
    })

// Style Match Recommendation
IF furniture_style matches room_style:
    recommendations.append({
        "type": "style",
        "message": f"Matches {room_style} style",
        "confidence": 85
    })

// Color Coordination Recommendation
IF furniture_color complements wall_color AND floor_type:
    recommendations.append({
        "type": "color",
        "message": f"{furniture_color} complements {wall_color} walls and {floor_type} floor",
        "confidence": 80
    })

// Lighting Recommendation
IF room_has_natural_light:
    recommendations.append({

```

```
"type": "lighting",  
"message": "Good natural lighting enhances furniture appearance",  
"confidence": 75  
})
```

Save rendered image:

```
preview_filename = f'realistic_ar_{uuid4()}.jpg"  
cv2.imwrite(preview_filename, img)
```

2. Return JSON response:

```
{  
  "ar_preview_url": f"/static/ar_previews/{preview_filename}",  
  "room_analysis": {  
    "dimensions": {width, depth, height},  
    "style": room_style,  
    "floor_type": floor_type,  
    "wall_color": wall_color,  
    "lighting": lighting_type  
  },  
  "recommendations": recommendations,  
  "message": "AR preview generated successfully"  
}
```

7.2 External APIs/SDKs

No APIs used till now.

7.3 User Interface

The user interface is developed using React.js and follows a simple, clean structure. The UI is designed to keep navigation easy and to allow users to move between modules without confusion.

7.3.1 Login Screen

A secure page where users enter their email and password to access the EZSell platform.



Welcome Back

Sign in to continue your selling journey

Username

Password

[Forgot password?](#)

Login

OR CONTINUE WITH

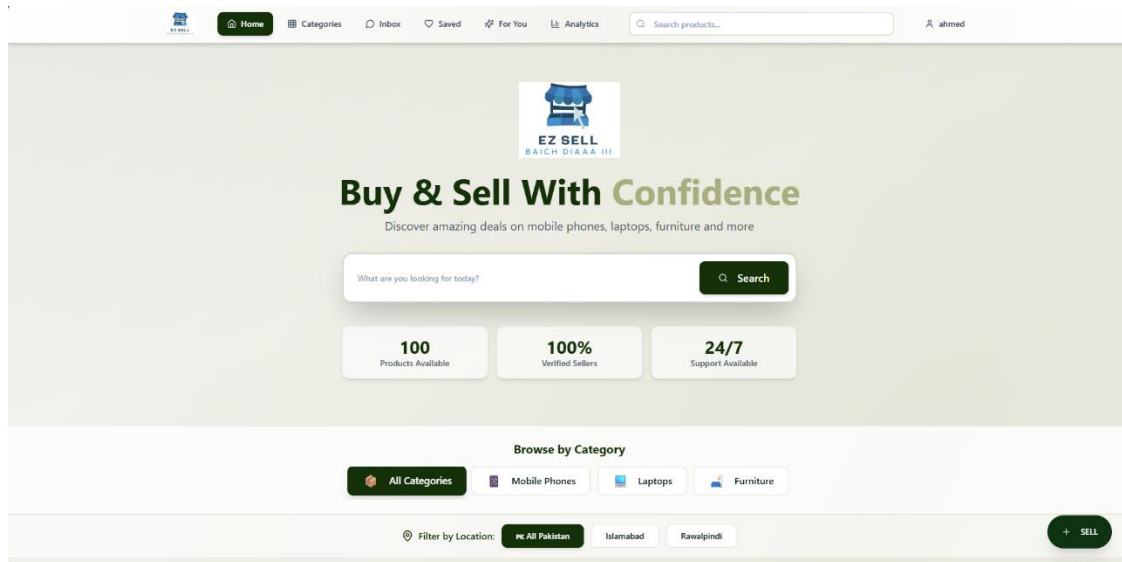
 **Continue with Google**

[Don't have an account? Sign up](#)

Figure 2 Login Screen

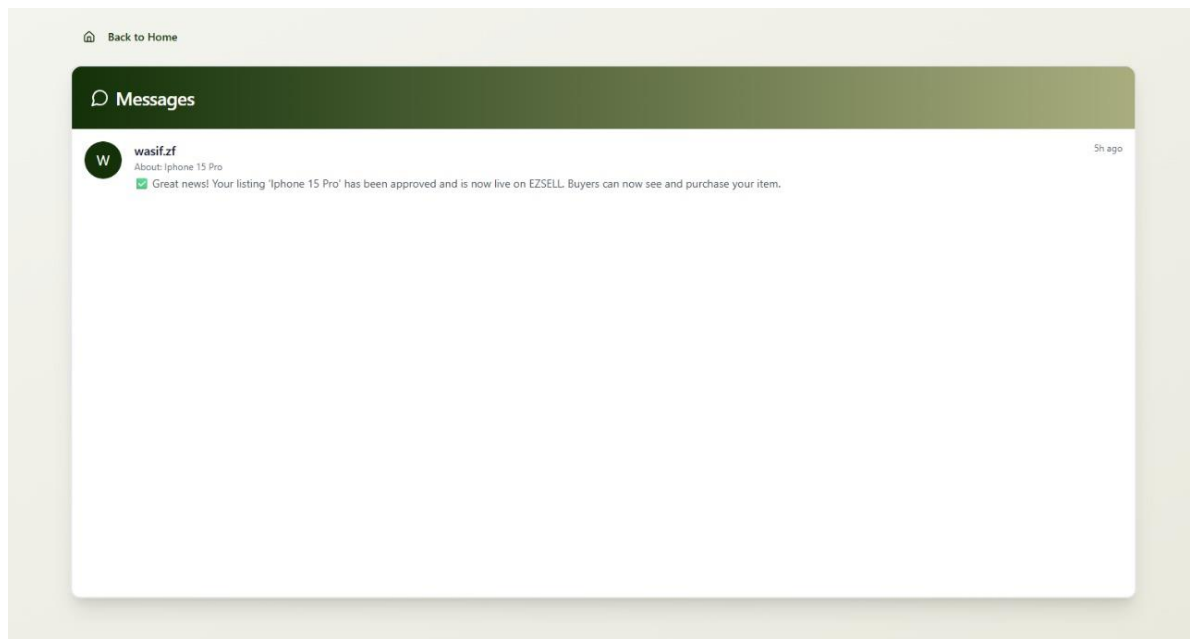
7.3.2 Home Screen

Displays all active listings, search options, and navigation shortcuts for users to explore products.



7.3.3 Messaging/Communication Screen

Provides a chat interface for buyers and sellers to communicate directly within the platform.



7.3.4 Post an AD with AI predictions Screen

Allows users to upload item details and receive an instant AI-generated price estimate before posting the listing.

Create New Listing

Add a new product with AI-powered price prediction

Step 1: Product Details

Category *

Mobile

Condition * (Affects Price Prediction)

5 - Average/Used (~60% value)

Select the condition level that best matches your product. Higher numbers = better condition = higher predicted price.

Title * (Include brand, model, specs)

IPHONE 7

Title is valid and contains relevant information

Description *

Condition

City *

Islamabad

Area/Sector *

F-10

Product Images (Up to 7 images)

1/7 images added

Remove All

JPG, JPEG, PNG • Max 10MB each • Up to 7 images

Step 2: Mobile Specifications

Brand *

Apple

RAM (GB)

Unknown

Storage (GB)

Unknown

Camera (MP)

Unknown

☐ 5G Support

☐ AMOLED Display

☐ Original Box

☒ PTA Approved

☐ Warranty

Step 3: AI Price Prediction

AI Predicted Price: **Rs 26,019**

Range: Rs 23,269 - Rs 28,769

Recommendation: Predicted price. Adjustments: +Rs 6,004 (PTA Approved) -Rs 13,343 (Average/Used, Level 5)

Tip: Adding RAM, Storage, and Camera specs can improve prediction accuracy

78% Confidence

Step 4: Set Your Price

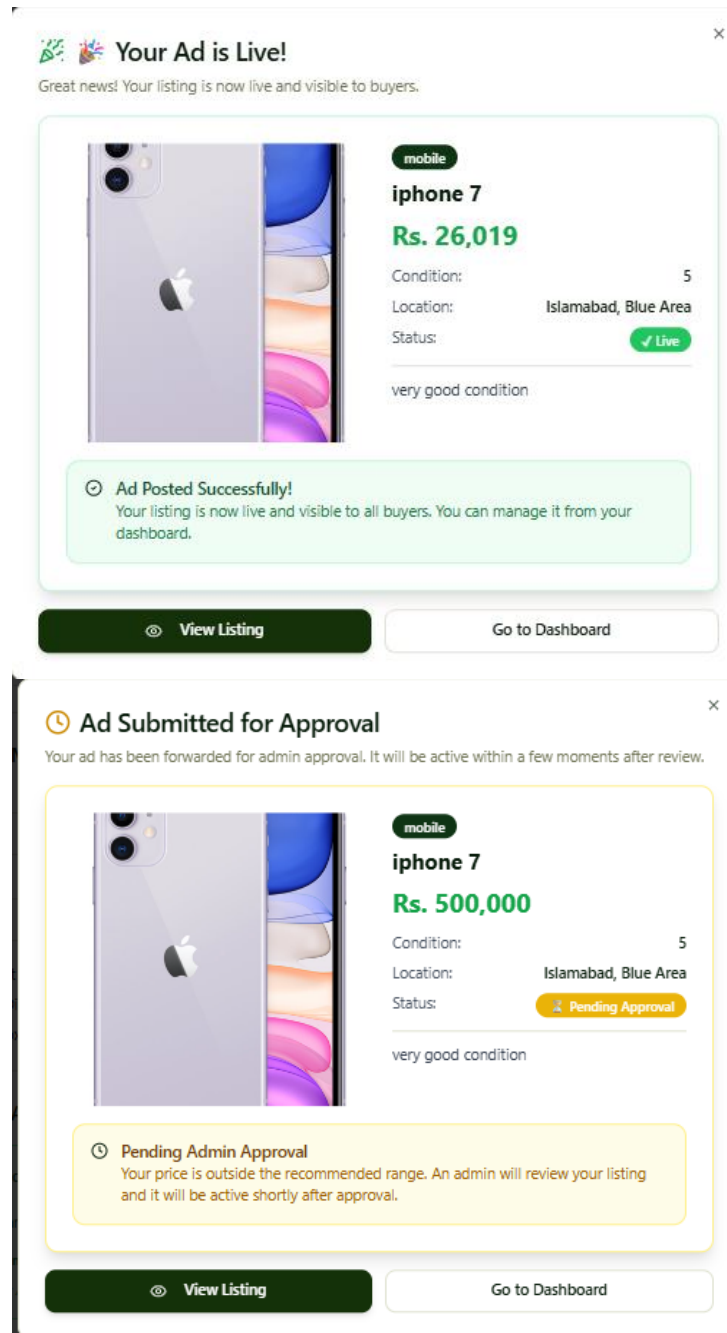
Your Price (PKR) *

26019

Recommended: Rs 26,019(±Rs 2,750)

Auto-Approval Range: Rs 21,019 - Rs 31,019

Create Listing



7.3.5 Ads Listings Dashboard

[Back to Home](#)

Dashboard

Welcome back, ahmed!

[Browse Listings](#) [Logout](#)

Total Listings: **163**

Active Listings: **163**

Total Views: **133**

Saved Favorites: **8**

[My Listings](#) [Favorites \(8\)](#) T1 [Most Recent](#) [+ Create Listing](#)

No image

Oppo Reno 13F 256GB PTA... Active

Rs 63,000 Oppo Reno 13F 256GB PTA Approved Main Boulevard Gulberg, Lahore•10 hours ago Call

PKR 63,000 0 views

No image

Samsung Galaxy 21 Ultra 8GB... Active

Rs 80,000 Samsung Galaxy 21 Ultra 8GB-128GB PTA Approved Mobile Township, Lahore•10 hours ago Call

PKR 80,000 0 views

No image

Samsung Galaxy S22 Approved... Active

Rs 64,999 Samsung Galaxy S22 Approved 8GB-128GB Allama Iqbal Town - Huma Block, Lahore•10 hours ago Call

PKR 64,999 0 views


king size bed Active

good condition

PKR 40,000 42 views


sofa set Active

abcdgr

PKR 35,000 22 views


2 seater sofa Active

kmnknkdscn

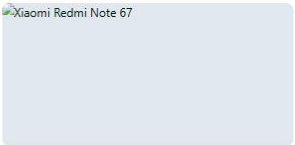
PKR 55,000 57 views


iphone 11 Pending

svmsjnsilomsxs

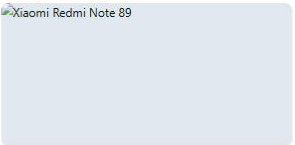
Under Review: Your listing is being reviewed by admin.

PKR 55,555 10 views


Xiaomi Redmi Note 67 Active

Brand new phone with warranty

PKR 45,000 0 views

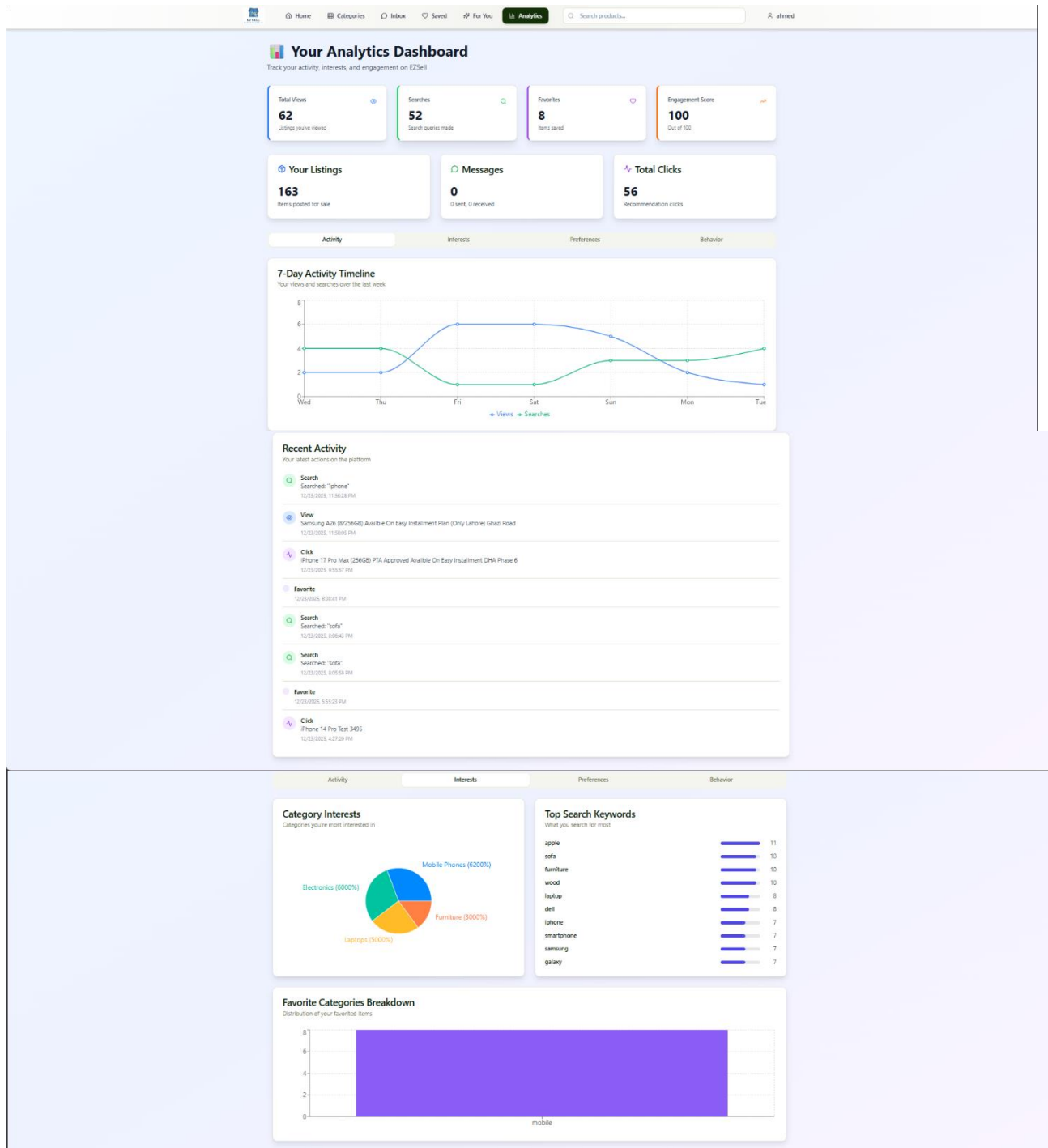

Xiaomi Redmi Note 89 Active

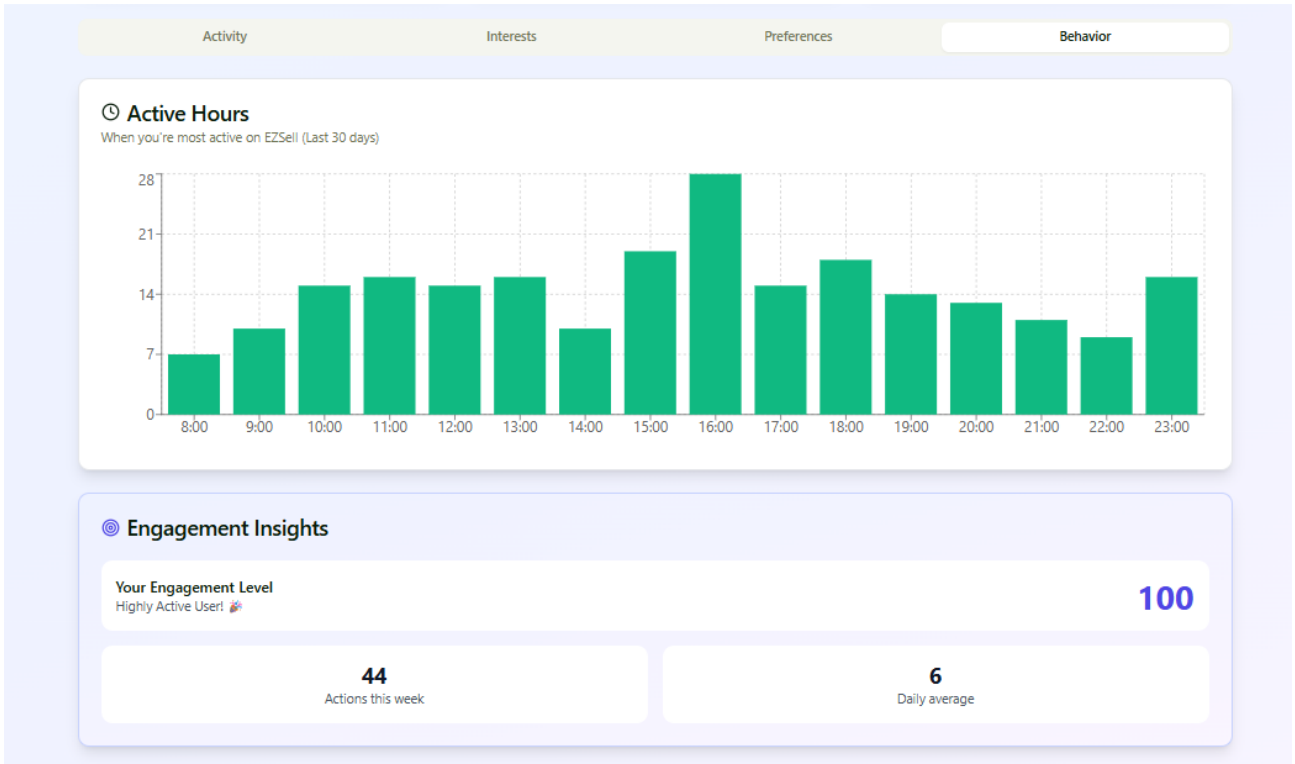
Brand new phone with warranty

PKR 45,000 0 views

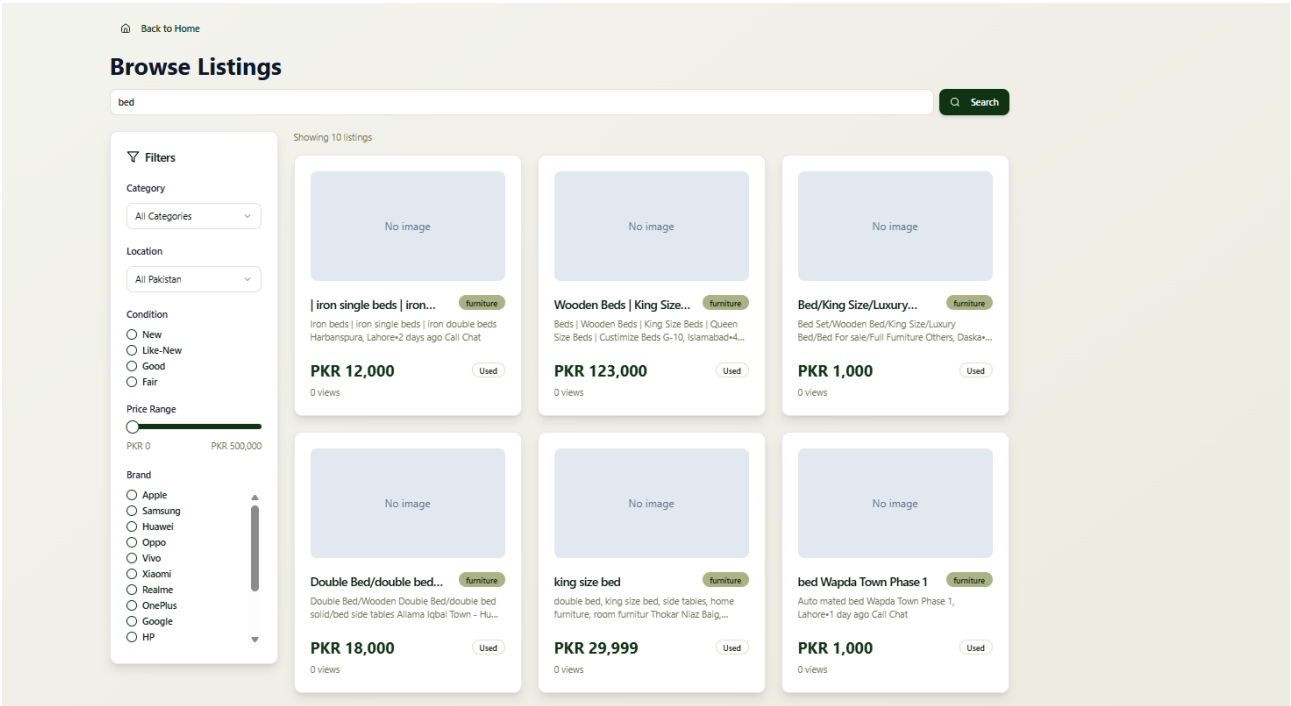
7.3.6 User Analytical Dashboard

Shows personalized insights such as user activity, views on their listings, and performance trends.

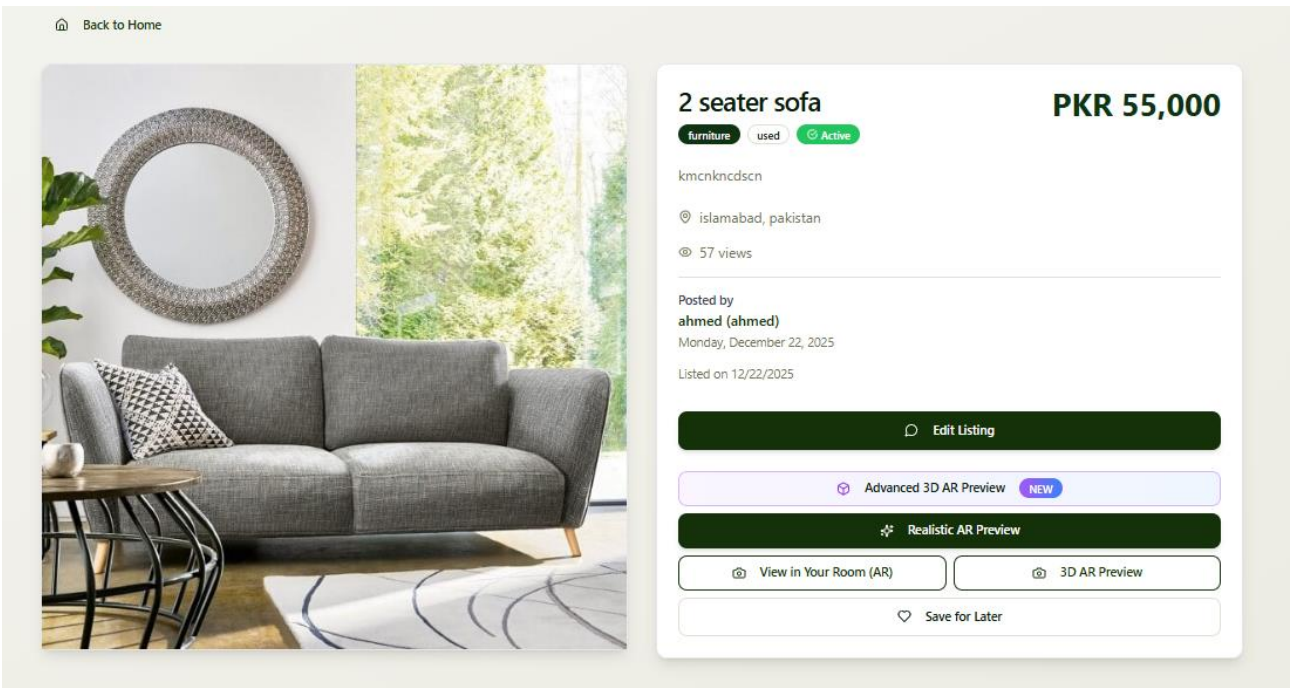


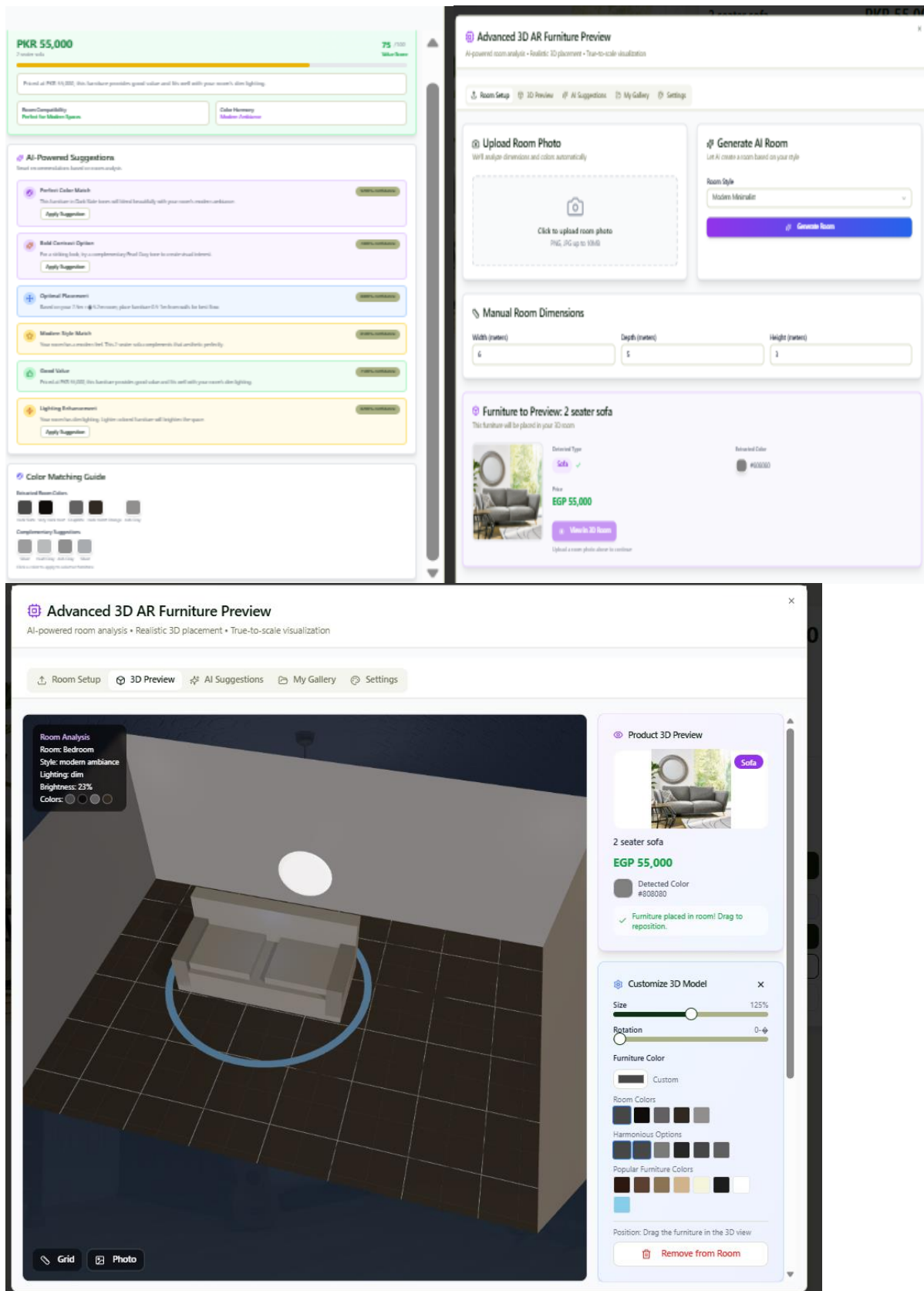


7.3.7 User Analytical Dashboard



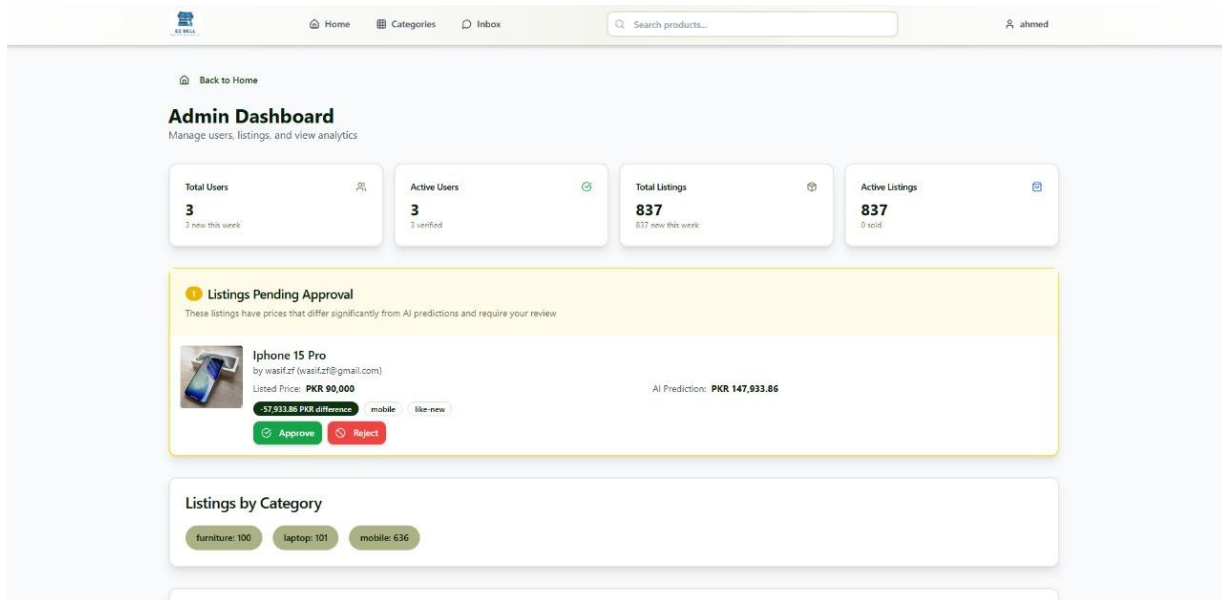
7.3.8 3D AR Furniture





7.3.9 Admin Analytical Dashboard

Displays system-wide analytics including total users, listings, category trends, and platform performance metrics.



7.4 Deployment

- The **EZSell platform** is deployed on a **local host** for development and testing. The backend is built using **FastAPI**, which handles REST APIs, business logic, authentication, AI price prediction endpoints, and AR processing requests.
- The frontend is developed using **React.js** and runs on a local Node.js development server, providing a responsive and interactive user interface.
- User data, product listings, AI predictions, AR session data, and analytics metrics are stored in a **PostgreSQL database hosted on Supabase**. Supabase also manages secure storage for images, listing photos, and AR assets.
- All components communicate over standard **HTTP/HTTPS protocols**. This local deployment environment allows developers to test, debug, and refine the application before deploying it to a cloud production server.

8 Testing and Evaluation

8.1 Deployment Environment (Local Testing Setup)

During development, we ran and tested the whole system on our local machines. The project had two main parts: the backend API and the frontend web application. Both were connected and tested together in a local setup.

8.1.1 Backend Environment

- **Framework:** *FastAPI*
- Runtime: Python 3.13
- Server Used: Uvicorn (development server)
- Hosting Mode: Localhost (<http://127.0.0.1:8000/>)
- Main Dependencies:
- fastapi, uvicorn, sqlalchemy, pydantic, httpx, python-jose, passlib[bcrypt], google-genai, python-multipart, python-dotenv, pandas, numpy, matplotlib, prophet, transformers, torch, groq, bcrypt

8.1.2 Frontend Environment (EZSell Project)

- Framework: React.js
- Runtime Environment: Node.js (LTS version)
- Local Hosting URL: <http://localhost:8080/>
- Package Manager: npm
- **Primary Dependencies:**
 - React Router (navigation)
 - Axios / Fetch API (backend API integration with FastAPI)
 - State Management (Redux / Context API)
 - UI Libraries (Tailwind CSS / Material UI)
 - Utility Libraries (e.g., date/time formatters, file upload handlers)

8.1.3 Testing Setup (EZSell Project)

- **Simultaneous Local Execution:**
Both the **FastAPI backend** and the **React.js frontend** were run together on the local development system.

- **API Communication:**

The frontend interacted with the backend using **REST API calls** over local URLs (e.g., `http://localhost:8000/api/...`).

- **Modules Tested in Local Environment:**

All system modules were tested end-to-end, including:

- **User Authentication & Profile Management**
- **Product Listings & Media Uploads**
- **AI Price Prediction** for mobiles, laptops, and furniture
- **User Analytics Dashboard**
- **Admin Analytics Dashboard**
- **3D AR Furniture Module**
- **Database CRUD Operations** on Supabase (PostgreSQL + Storage)

8.2 Unit Testing

Unit testing checks individual components to ensure each one behaves as expected.

8.2.1 Unit Testing 1: Validate login functionality with correct and incorrect inputs.

Testing Objective: To ensure the login form is working correctly with valid and invalid inputs.

Table 8.2.1: Unit Testing 1 login Functionalities

No.	Test case	Attribute and value	Expected result	Result
1	Check valid login	username: ahmed Password: ahmed123	User is logged in and redirected to dashboard.	Pass
2	Invalid email format	username: ahme	Error, no such sort of username	Pass
3	Invalid password	Password: EZsell123	Dashboard inaccessible, login page refresh	Pass

8.2.2 Unit Testing 2: Listing Image Upload

Testing Objective: Ensure supported file types are accepted and invalid formats are rejected.

Table 8.2.2:

No.	Test Case	Attribute and Value	Expected Result	Result
1	Upload supported image	image.jpg	File accepted and uploaded to storage	Pass
2	Upload unsupported format	file.txt	Wont show any other sort of file other than jpg, jpeg, png	Pass
3	Upload large file	big_image.webp (10MB)	Error	Pass

8.2.3 Unit Testing 3: Price Prediction

Testing Objective: Ensure that the AI price prediction module correctly processes features and returns a valid predicted price with confidence.

Table 8.2.3

No.	Test Case	Attribute and Value	Expected Result	Result
1	Predict price for mobile	title=Samsung s23, condition=used	Return predicted price + confidence	Pass
2	Predict price for laptop	brand=HP omen, core i3, RAM=8GB	Return predicted price + confidence	Pass
3	Missing required feature	condition = null	No Prediction	Pass

8.2.4 Unit Testing 4: Listing Creation (Post Ad)

Testing Objective: Ensure that new listings are created successfully with all required fields.

Table 8.2.4

No.	Test Case	Attribute and Value	Expected Result	Result
1	Valid listing	title, desc, price, category, image provided	Listing saved successfully	Pass
2	Missing title	title=null	Error	Pass
3	Price not numeric	price="ten thousand"	Error	Pass
4	Image missing	no images uploaded	Error	Pass

8.2.5 Unit Testing 5: Listing Creation (Post Ad)

Testing Objective: Ensure that new listings are created successfully with all required fields.

Table 8.2.5

No.	Test Case	Attribute and Value	Expected Result	Result
1	Fetch dashboard data	user_id valid	Returns totals (views, listings, contacts)	Pass
2	No listings	user with 0 listings	Show zero analytics	Pass
3	Price not numeric	price="ten thousand"	Error	Pass
4	Invalid user_id	user_id = null	Error	Pass

8.2.6 Unit Testing 6: Admin Analytics Dashboard

Testing Objective: Verify correct aggregation of platform-wide metrics.

Table 8.2.6

No.	Test Case	Attribute and Value	Expected Result	Result
1	Fetch system metrics	none	Return totals: users, listings, active listings	Pass
2	No users available	empty database	Display zero metrics, no errors	Pass

8.2.7 Unit Testing 7: AR Room Analysis Functions

Testing Objective:

Ensure that AR room analysis helper functions correctly process images and detect room properties.

No.	Test Case	Attribute and Value	Expected Result	Result
1	Detect dominant colors	room image with blue walls	Returns color: "Blue", hex: "#4A90E2"	Pass

No.	Test Case	Attribute and Value	Expected Result	Result
2	Estimate room dimensions	room image with visible walls	Returns width, height, depth estimates	Pass
3	Detect room style	modern furniture in image	Returns style: "Modern", confidence: 85%	Pass
4	Calculate suitability score	furniture vs room dimensions	Returns score: 75–95%	Pass
5	Invalid image format	file: not an image	Returns error: "Invalid image"	Pass

8.3 Functional Testing

The functional testing will take place after the unit testing. In this functional testing, the functionality of each of the module is tested. This is to ensure that the system produced meets the specifications and requirements.

8.3.1 Functional Testing 1:

Testing Objective:

Ensure that uploaded listing images/files are accepted for supported formats and rejected otherwise.

No.	Test case/Test script	Attribute and value	Expected result	Actual result	Result
1.	Upload valid image	image.jpg	File accepted and uploaded to storage	Uploaded successfully	Pass
2.	Upload unsupported file type	file.txt	Won't allow	Didn't allowed	Pass
3.	Upload large file (>10MB)	Big image.webp	Display warning: "File size too large"	Warning shown	Pass

8.3.2 Functional Testing 2: AI Price Prediction

Testing Objective:

Verify that the price prediction engine returns valid prices and confidence for different listings.

No.	Test Case / Test Script	Attribute and Value	Expected Result	Actual Result	Result
1	Predict price for mobile device	Category: Mobile, Brand: Samsung, RAM: 8GB, Storage: 128GB	Display predicted price along with confidence range	Prediction displayed correctly	Pass
2	Predict price for laptop	Category: Laptop, Brand: HP, Processor: Intel i5, RAM: 8GB	Display predicted price along with confidence range	Prediction displayed correctly	Pass
3	Prediction with high confidence	Well-defined and complete specifications	Display narrow confidence range	High confidence shown correctly	Pass
4	Auto-approval range display	Successful price prediction	Display green auto-approval price range	Auto-approval range displayed correctly	Pass
5	Re-prediction on field change	User modifies product specifications	System automatically triggers a new price prediction	Auto-prediction working correctly	Pass

8.3.3 Functional Testing 3: User & Admin Dashboard Analytics

Testing Objective:

Ensure that dashboards correctly display metrics for users and admins.

No.	Test Case / Test Script	Attribute and Value	Expected Result	Actual Result	Result
1	User dashboard with listings	Valid user_id with existing listings	Display total listings, total views, and contact count	Metrics displayed correctly	Pass
2	User dashboard with no listings	User with zero listings	Display message: "No data available"	Message displayed correctly	Pass
3	Admin dashboard metrics	Complete system data available	Display total users, total listings, and active listings	Metrics displayed correctly	Pass
4	Admin dashboard with empty database	No users or listings in system	Display zero values for all metrics	Zero metrics displayed correctly	Pass
5	Admin dashboard top sellers	Users with multiple listings	Display top 5 sellers ranked by number of listings	Top sellers displayed correctly	Pass

8.3.4 Functional Testing 4: Listing Creation & Validation

Testing Objective:

Ensure that listing creation, duplicate prevention, and price validation work correctly.

No.	Test Case / Test Script	Attribute and Value	Expected Result	Actual Result	Result
1	Create listing with valid data	Title, description, price, category, images provided	Listing created successfully and user is redirected to dashboard	Listing created and visible on dashboard	Pass
2	Create listing with duplicate title	Title already exists (case-insensitive)	System rejects submission with error: "A listing with this title already exists"	Duplicate title rejected correctly	Pass
3	Create listing with duplicate images	Same image uploaded twice	System rejects submission with error: "Duplicate image detected within upload"	Duplicate image rejected	Pass

No.	Test Case / Test Script	Attribute and Value	Expected Result	Actual Result	Result
4	Create listing with alphabetic price	Price entered as “five thousand”	System rejects submission with error: “Price must be a positive number. Alphabetic prices are not allowed”	Alphabetic price rejected	Pass
5	Create listing with valid images	1–7 images, JPG/PNG format, each under 10MB	Images uploaded successfully	Images uploaded and displayed correctly	Pass

8.3.5 Functional Testing 5: Admin Approval System

Testing Objective:

Ensure that the administrator can correctly approve, reject, and manage pending listings

No.	Test Case / Test Script	Attribute and Value	Expected Result	Actual Result	Result
1	View pending listings	Admin logged in	Display all listings with status set to “Pending”	Pending listings displayed correctly	Pass
2	Approve listing manually	Admin clicks Approve button	Listing status changes to “Active” and notification is sent to seller	Status changed to active successfully	Pass
3	Reject listing manually	Admin clicks Reject and provides reason	Listing status changes to “Rejected” and rejection reason is saved	Listing rejected with reason saved	Pass
4	Auto-approval for listings within range	Price within $\pm 20\%$ of predicted price	Listing is automatically approved without admin intervention	Auto-approved successfully	Pass
5	Manual review for listings outside range	Price outside $\pm 20\%$ of predicted price	Listing is sent to admin queue for manual review	Listing sent to pending review correctly	Pass

8.3.6 Functional Testing 6: 3D AR (Augmented Reality) Features

Testing Objective:

Ensure that 3D AR model upload, preview, customization, and mobile viewing work correctly for furniture listings.

No.	Test Case / Test Script	Attribute and Value	Expected Result	Actual Result	Result
1	Upload room photo for AR analysis	Valid JPG/PNG image, furniture listing	Room analyzed for dimensions, style, lighting; AI insights displayed	Room analysis working correctly	Pass
2	AR canvas preview with furniture	Room photo uploaded	Canvas viewer opens; furniture rendered as 3D rectangles with proper perspective	Canvas preview working correctly	Pass
3	AR customization (color/material)	Change furniture material (wood, leather, fabric, metal)	Furniture color/material updates in real-time on canvas	Customization working correctly	Pass
4	Drag and drop furniture placement	Drag furniture objects on canvas	Furniture repositions with shadows and perspective maintained	Drag-and-drop working correctly	Pass
5	Multiple view angles	Switch between front/3D/top views	Canvas updates to display different perspective angles	View switching working correctly	Pass

8.4 Business Rules Testing

Testing Objective:

Verify that business logic rules are correctly applied for listings, price predictions, and anomalies.

- **A1:** Flag duplicate listing
- **A2:** Mark unusually high price
- **A3:** Flag outdated listing
- **A4:** Flag invalid data entry
- **A5:** No anomaly

Condition / Rule	R1	R2	R3	R4	R5
Listing Title Same?	Yes	No	No	No	No
Price unusually high?	Yes	Yes	No	No	No
Very old listing?	No	No	Yes	No	No
Invalid Input / Data Error?	No	No	No	Yes	No
Action	A1	A2	A3	A4	A5

8.5 Integration Testing

Integration testing ensures that different modules work correctly when connected together.

8.5.1 Integration Testing 1: User Registration - Email Verification - Login Testing

Objective:

Verify that the user registration, authentication, and initial dashboard flow are correctly integrated.

No.	Test Case / Test Script	Test Attribute Value	and Expected Result	Actual Result	Result
1	User registers and account is created	Valid username, email, and password	User account is created in the database and a success message is displayed	Account created successfully	Pass
2	User registers with duplicate username	Username already exists	System displays error message: "Username already exists"	Duplicate username rejected	Pass
3	Newly registered user logs in	Valid newly registered credentials	User logs in successfully and is redirected to the user dashboard	Login working correctly	Pass
4	Newly registered user views empty dashboard	New user with no listings	Dashboard displays "No listings yet" message	Empty dashboard state working correctly	Pass

8.5.2 Integration Testing 2: Listing Upload - AI Price Prediction - Dashboard Update

Testing Objective:

Verify the end-to-end flow from uploading a listing to predicting price and updating dashboards.

No.	Test Case / Test Script	Attribute and Value	Expected Result	Actual Result	Result
1	Full listing pipeline	User uploads a new listing	Data is saved successfully, price is predicted, and dashboards are updated	End-to-end process worked correctly	Pass
2	Auto-approval triggered by price prediction	Listing price within approved range	Price predicted → listing auto-approved → listing becomes active in feed	Auto-approval working correctly	Pass
3	Manual review triggered by price prediction	Listing price outside approved range	Price predicted → listing sent to admin → status set to pending	Manual review triggered successfully	Pass
4	User dashboard updates after listing creation	User creates a new listing	Listing count increases on user dashboard	User dashboard updated correctly	Pass

No.	Test Case / Test Script	Attribute and Value	Expected Result	Actual Result	Result
5	Admin dashboard updates after listing creation	New listing added to system	Total listings count increases in admin analytics	Admin dashboard updated correctly	Pass

8.5.3 Integration Testing 3: Login - Dashboard - Listings

No.	Test Case / Test Script	Attribute and Value	Expected Result	Actual Result	Result
1	User logs in and dashboard loads	Valid credentials	Dashboard loads successfully with listings feed and “Post Ad” button visible	Dashboard loaded correctly	Pass
2	User clicks “Post Ad”	Logged-in user	User is able to create and upload a listing if they choose to post	Listing upload works correctly	Pass
3	Buyer-behavior user ignores “Post Ad”	User behaving as buyer only	User can browse and scroll listings without being required to post	Browsing works correctly	Pass
4	User views listings	Any logged-in user	User can view all available listings	Listings displayed correctly	Pass

8.5.4 Integration Testing 4: Category Selection - Form Fields - Price Prediction

Testing Objective:

Verify that dynamic form fields update correctly based on category selection and that price prediction works accordingly.

No.	Test Case / Test Script	Attribute and Value	Expected Result	Actual Result	Result
1	User selects Mobile category	Category set to “Mobile”	Brand, RAM, storage, and condition fields are displayed	Mobile-specific fields displayed correctly	Pass
2	User enters mobile specifications	Brand: Apple, RAM: 8GB, Storage: 256GB	Predicted price is displayed along with confidence range	Price prediction working correctly	Pass
3	User selects Laptop category	Category set to “Laptop”	Processor, GPU, RAM, and storage fields are displayed	Laptop-specific fields displayed correctly	Pass
4	User changes category selection	Switch from Mobile to Laptop	Mobile fields are hidden and laptop fields are displayed	Fields updated correctly	Pass

No.	Test Case / Test Script	Attribute and Value	Expected Result	Actual Result	Result
5	User completes specifications	All required fields filled	Green auto-approval price range is displayed	Auto-approval range displayed correctly	Pass

8.5.5 Integration Testing 5: Favorites System - Dashboard - Listing Updates

Testing Objective:

Verify that the favorites system is correctly integrated with the user dashboard and listing updates.

No.	Test Case / Test Script	Attribute and Value	Expected Result	Actual Result	Result
1	User adds a listing to favorites	User clicks favorite icon on a listing	Favorite is saved to database and icon changes to active (filled) state	Favorite saved successfully	Pass
2	User adds favorite and dashboard updates	Listing added to favorites	Favorite count increases on user dashboard	Dashboard updated correctly	Pass
3	User views favorites	User navigates to favorites page	All favorited listings are displayed with complete details	Favorites displayed correctly	Pass
4	User removes a favorite	User clicks unfavorite icon	Favorite is removed, icon changes to unfilled state, and count decreases	Favorite removed successfully	Pass
5	Listing deletion updates favorites	Seller deletes a listing	Listing is automatically removed from all users' favorites	Favorites cleanup working correctly	Pass

8.5.6 Integration Testing 6: 3D AR Model Upload → Storage → Preview

Testing Objective:

Verify the end-to-end 3D AR workflow, including model upload, storage, listing display and preview.

No.	Test Case / Test Script	Attribute and Value	Expected Result	Actual Result	Result
1	Upload 3D model and store on server	Upload GLB file for furniture	Model saved to ar_previews directory and AR badge shown on listing	Model stored successfully	Pass
2	Listing created with 3D AR model → badge displayed	Furniture listing with uploaded 3D model	“AR View” badge visible on listing card in feed	Badge displayed correctly	Pass
3	User clicks AR preview → 3D viewer loads	Click “View in AR” button	AR viewer opens; 3D model renders with rotate/zoom controls	AR viewer working correctly	Pass
4	User customizes AR model → changes saved	Change color/material and save	Customizations saved and reflected in all AR views	Customizations saved successfully	Pass